

ITE 1943 – ICT PROJECT

PROJECT REPORT

LEVEL 01

Personalized meal and workout planning system

Submitted by:

Dulara Chamika Bandara

E2410352

Bachelor of Information Technology (External Degree)

Faculty of Information Technology

University of Moratuwa

Table of Contents

1.Introduction.....	1
2.Related Work	4
3.Systems Analysis	8
4.System Design.....	13
5.System Implementation.....	27
6.Appendix.....	45
7.References	87

1.Introduction

In the modern hectic and busy lifestyle, it is so hard to balance classes, work and social life, not to mention we students who are more than eager to remain healthy. Use of manual, far-between methods of tracking food and workouts is only going to complicate achievement of fitness targets. That is why I am enthusiastic about the Personalized Meal and Workout Planning System, an application on the computer that will help to combine everything into a single convenient and simplified mechanism to monitor our health.

1.1Background & Motivation

The greatest problem I am addressing is the difficulty that individuals like us have in maintaining a healthy lifestyle when the old ways are simply no longer applicable.

The broken system we are stuck in brings the key problems:

- i. **Strenuous labor and time:** Calories, diets and workout organization take the eternity and cannot be included to the schedule of a student.
- ii. **Inability to integrate:** Existing apps tend to be nutrition-oriented (such as MyFitnessPal) or activity-oriented (such as Strong or Google Fit), never both at the same place with so many features.
- iii. **Complexity:** The apps may be overwhelming and may have too many features that only inundate students who require a simple, hands-free solution.

Impact on Relevant Users ;

Such time and workflow challenges prevent us to have a consistent and organized perspective on health that will provide us with consistent results instead of intermittent ones

Justification for a Computer-Based Solution (Motivation)

The motivation factor in this project is the necessity of an effective and easy to use tool that minimizes on the hassle. The Personalized Meal and Workout Planning System can:

- i. **Automate complicated operations:** It spares us of tedious calculations, manual data entry and does all this digitally.
- ii. **Provide a one-stop solution:** It combines the meal planning, tailored workout plans, and tracking of the progress into a single easy app.
- iii. **Keep things consistent:** Since automation provides consistency, it is easy to maintain healthy habits compared to managing two separate tools.

1.2 Problem in brief

What we are actually trying to correct with this project is the hard work that we have to go through in an environment full of chaos, time-draining circumstances in order to remain healthy. The modern routine requires excessive documentations and different diets and exercise schedules and sets up a difficult obstacle to permanent adherence. The market does not have one clean cut, simple application that integrates weekly meal plans and customized fitness programs in a format that is truly student friendly.

It is the Personalized Meal and Workout Planning System, a simple desktop application. This is a one-stop device that is specifically integrated to enable students and similarly-minded individuals to have one place where they can plan their meals, come up with their individual workout timetables, and even monitor their progress. The centralization and automation of these fundamental activities by the system simplify by far the way we construct and maintain the healthy habits in a consistent manner.

1.3 Aims & Objective

The objective and goal of this project establishes the final objectives as well as the actual steps that should be undertaken in order to design the Personalized Meal and Workout Planning System. It is expected to produce a convenient desktop application that will address the issue of inadequate and random health-management practices of university students. The app will be a unified tool through which we can plan our meals, create our own workout programs and monitor our overall progress, providing us with a convenient means of ensuring we have our diet and exercise programs on track.

In order to pin down this objective, we must strike the following objectives:

- i. Critical Analysis of System Requirements - We will break down the functional and non functional requirements into details.
- ii. Database Development - Develop a robust database that will handle information about food, recipes, exercises and user data.
- iii. Meal Planning Feature Implementation – Develop a feature that will allow the user to prepare a unique weekly meal plan and automatically displays the overall daily nutrition.
- iv. Creation of Workout Scheduling Feature -Create a workout planning app that allows people to create a personal workout plan by selecting the exercises in a list.

- v. Progress Tracking Functionality – Introduce the means of tracking and following the process of their health progress over time
- vi. User Interface Design – Design a clean, easy to use UI that is comfortable and user-friendly to all functions.
- vii. Design and Develop the System's core engine and functionality This is an outline of the architecture and implementation of the core functionality as a fully working desktop application.
- viii. Testing the Proposed System Performance, usability and efficiency of solving the problem stated.

1.4 Summary

This introductory chapter preconditions Personalized Meal and Workout Planning System. We identified the primary issue: students of universities having problems with inconsistent health-management practices due to manual practices being inefficient and current fitness apps being disconnected. The rational of an integrated solution based on a computer is aimed at automating and facilitating uniformity. Last but not the least, the chapter had the particular purpose mentioned in an all-in-one desktop application and outlined the objectives required to succeed.

We will then continue on this platform by evaluating such systems that are already in existence. Such a comparative analysis will also demonstrate the advantages and disadvantages of competitors and support the argument of our original design.

2.Related Work

The next chapter critically examines existing health and fitness management systems and describe the problems we are addressing and why we are developing our Personalized Meal and Workout Planning System.

The primary problem addressed by the project is the difficulty of university students to live a healthy life due to a handful of essential problems:

- i. **Time-consuming and Unorganized** - Calories and diets; most of the time the workouts are not automated, and it is almost impossible to track calories and diets daily and still adhere to the fitness plan, thus eventually giving up on them.
- ii. **Digital Tools Fragmentation**: Apps that are currently available are either nutritional or general activity tracking with very little integration of both into a single, convenient app. This compels it on the users to manage numerous systems, which makes the process complicated and cumbersome.
- iii. **Difficulty of the Existing Solutions** - It is full of complicated interfaces and features that most existing solutions are not appropriate to students, who require a simple integrated desktop solution.

There are currently available systems and solutions applicable in operations management research. There are the existing systems and solutions that can be used in operations management research.

In order to further contextualize our present environment and to justify our solution, we analyzed five significant existing systems, with each having a varying main focus.

[1]MyFitnessPal (Under Armour)

MyFitnessPal is a popular mobile/web-based and calorie and macronutrient tracking app.

- Its approach to dealing with the problem: The enormous food database and barcode scanner is its biggest advantage, as it makes tracking food easily and effectively with it. It provides simple meal recording and is short in terms of systematic meal planning.
- Limitations: It is primarily a meal tracking app and does not generalize with the rest of fitness planning significantly

[2]Strong (Strong Inc.)

Strong is an advanced exercise application on mobile.

- How it deals with the issue: It is outstanding in the exercise side of things. Its functions are based on developing personal training programs, an in-depth weight-lifting program, and the in-built application with a plate calculator and timer to plan great workouts.
- Limit: none, no food, no calories or a meal, just exercise.

[3]Google Fit (Google LLC)

Google Fit primarily forms an activity aggregator both on mobile and Web.

- *The way it addresses the issue: It monitors physical activity data such as steps, distance, and heart rate, and retrieves the data connected with the different fitness gadgets.
- Limitations Focused on activity tracking, no meal planning or nutritional functionality.

Cronometer (Cronometer software, Inc.)

Cronometer is a multi-purpose nutrition app that is accessible both on mobile and web.

- How it addresses the problem: It provides in-depth food diaries and monitors up to 82 micronutrients - it is a high-complexity instrument of comprehensive nutritional tracking.
- Restriction: Like MyFitnessPal, it is more of a meal logging application and is highly complicated, which may discourage users who only require a basic operation

Lose It! (FitNow, Inc.)

This is the basic weight-loss application that is phone and browser-based, Lose It!

- What it does to deal with the issue: it allows you to track food and has a basic calorie budget system, which makes calorie counting not a complex task.
- Shortcoming: the planning is rather simplistic and cannot be combined with other health-related things - it is all about calories.

Comparative Analysis

<u>Feature</u>	<u>MyFitnessPal</u>	<u>Google Fit</u>	<u>Cronometer</u>	<u>Lose It!</u>	<u>Strong</u>
<u>Main Focus</u>	Nutrition & Calorie Tracking	Activity Tracking & Aggregation	Micro/Macro Tracking	Simple Calorie Counting	Workout Logging
<u>Meal Planning/Creation</u>	Logging Only	None	Logging Only	Logging Only	None
<u>Workout Planning/Creation</u>	Basic Templates	Activity Tracking Only	Basic Logging	Very Basic	Excellent
<u>Integration (Meal & Fitness)</u>	Low	Low	Low	Low	None
<u>Complexity</u>	Moderate	Simple Dashboard	High	Simple	Moderate
<u>Platform</u>	Mobile/Web	Mobile/Web	Mobile/Web	Mobile/Web	Mobile
<u>Tech Stack</u>	React / React native	Android Wear OS	Cross-Platform Mobile	Swift kotlin	React Native flutter

The Results of the Comparison

The study demonstrates that a more basic, all-in-one health planner is not filled:

- i. Complicated and disjointed: applications such as Cronometer are slightly too much, or you need to use several of them (MyFitnessPal to track food, Strong to do exercises).
- ii. There is the lack of all-in-one planning: there is no application that actually combines meal planner with personally tailored workouts in a convenient manner. They only have what you ate which is not going to assist in creating a future weekly plan.

Justification of the proposed system

The primary victory of our Personalized Meal and Workout Planner is that it has been designed directly around college students to address the fragmentation and complexity identified above.

- i. One solution fits all: in comparison to other applications that can hardly be synced, ours is a straight forward desktop application that approaches the ordeal of weekly meal planning, the development of your own workouts, and tracking progress in a single platform.
- ii. Planning over logging: another area that other apps lack is the planning gap, most of them only record what you did.
- iii. Uncomplicated and convenient: not complex and full of features like some apps, it is simple and straight forward, and as a result, much easier as a student to maintain healthy behaviors in the long run.

3.Systems Analysis

This chapter details the systematic analysis of the requirements for the Personalized Meal and Workout Planning System. It covers the functional requirements, which define what the system must do, and the non-functional requirements, which specify the system's quality attributes and performance constraints. Clear identification and analysis of these requirements ensure the system is developed within scope and effectively solves the problem of disorganized health management among university students.

3.1 System Requirements

The system requirements define all that the Personalized Meal and Workout Planner must accomplish to be a good fit. We separated them in to what (functional) and how well (non-functional). These are based on the ambition of producing a student-friendly desktop application.

i. Overview the Functional Requirements

Functional requirements describe what you can do in the system primarily. It must encompass all the important planning, tracking, and managing activities - such as signing in, further planning of your diet and workouts, and how to monitor your progress.

ii. Overview of Non-Functional Requirements

Non-functional requirements are used to establish standards regarding how the system is functional and the design limits and its construction. In the case of students, it is more about convenience, speed and safety. They determine the user experience and general quality.

3.2 Functional Requirements

The functional section enumerates the precise tasks and services that have to be provided to you by the system, which is in line with the purpose of an integrated plan-and-track facility.

User Management

- i. This core must maintain a safe and personal access.
- ii. The application allows the creation of a personal profile.
- iii. You are able to use your username/ password to log in.
- iv. Passing access through the system verifies your credentials with the database.
- v. Once logged in the profile loads and the dashboard appears.
- vi. You are able to add or remove your profile (CRUD)

Meal Planning & Selection

- i. This option allows you to create a sound meal plan - which other apps lack.
- ii. You are able to search, choose and add recipes to every meal slot within a selected date.
- iii. The application automatically counts and displays the sum total of nutrients in meals of a day.
- iv. You are able to save or post your daily plan.
- v. You are able to create, read, modify or delete meal plans (CRUD).

Workout Planning & Selection

- i. This section allows you to create custom workouts which are included in the fitness trackers but omitted in the full-stack flier apps.
- ii. Find and select entries in the database.
- iii. Establish the number of sets to do in each exercise.
- iv. Create a personal workout plan.
- v. Before saving, you have to name the routine.

Progress Logging

- i. Recording the progress is important; this enables you to trace more than planning in history.
- ii. You are able to indicate meals as finished.
- iii. You are also able to check workouts as complete.
- iv. The completions are all stored in a database history

Progress Visualization

- i. To ensure you continue being motivated, tracking data is clearly displayed in the system.
- ii. On the dashboard, you can see your progress in graphs and charts

3.3 Non-Functional Requirements

Non-functional requirements inform you about the quality bits that will make the app great particularly to our busy students

Usability

- i. The application must be easy and user-friendly, no tinkering about as with other applications.
- ii. Simple navigation: UI can be intuitive without being techno-savvy.
- iii. Appearance is also important - harmonized color, clean lines.
- iv. Stress-free UX: easy to use, all functions.

Performance

- i. As far as I have come to understand, the system should be able to react quickly to user movements in order to avoid losing people to impatience and frustrations.
- ii. Speed of loading: I believe that the system must take a short duration to load such as a few seconds.
- iii. Responsiveness: It must provide us with a quick response when we tap or click particularly when we plan or log.
- iv. Customization: It being a desktop app, it should be compatible with a variety of operating systems and screen resolutions.

Reliability

- i. The system must remain steady and consistent particularly when handling sensitive information.
- ii. Stability: It cannot be crashing especially when we are saving or updating information.
- iii. Data Integrity: The application should be able to maintain data integrity on critical operations.
- iv. Recovery: I would like it to save or backup automatically at the most critical times in order to save all the data.

Security

- i. The health information of the users and their personal credentials must be secured against unauthorized access.
- ii. Data Protection: The system must protect the information of users to the wrong hands.
- iii. Authentication: Username and passwords must be kept in a safe place and authenticated using a secure system.

Summary

I have in chapter 3, decomposed the system requirements of our Personalized Meal and Workout Planning System. The Functional Requirements are distributed into five key subnames: User Management, Meal Planning and Selection, Workout Planning and Selection, Progress Logging, and Progress Visualization and all are based on sound CRUD functionality. These specifications define what the system must be doing to achieve the project objective of integrated health management. The Non-Functional Requirements cement the quality features we require: it must have a huge Usability, it must be blisteringly fast Performance, it must be rock-solid Reliability, and it must be rock-solid Security to present to the users the finest of the finest experiences.

The following chapters will move a notch higher and provide a solution to this analysis and the subsequent chapters will give the design and implementation route of how we will achieve all these specifications.

4.System Design

This chapter moves from defining what the system must do to designing how it will work.

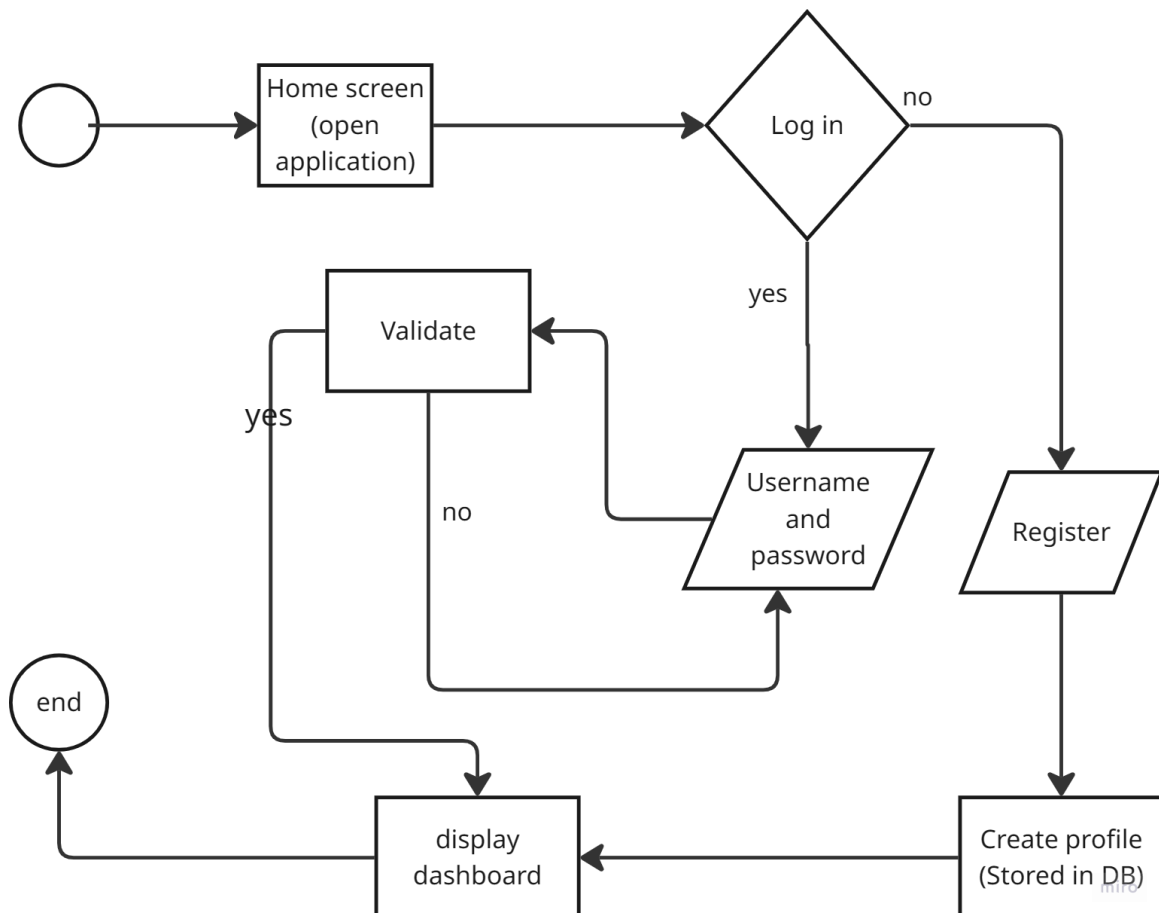
We will create a technical blueprint to guide the implementation. This chapter specifies the system's design. We will break down each requirement into clear, logical steps using two methods:

- Flowcharts to diagram the user's path and how data moves through the system.
- Pseudocode to write the step-by-step logic for operations like calculating nutrition or saving a workout.

For each model, we will state its inputs, the steps it follows, and its outputs. This detailed planning is essential for building a system that is reliable and easy for students to use.

4.1 Flow Charts

(1). User Management (Create Profile & Login)



1. Start and Initial Check

- Start/Input: The process begins with the user opening the application, which lands them on the Home screen.
- Process (Decision): The system immediately performs a check: 'Is user logged in?'.
 - Yes: If an active session or stored credential exists, the flow bypasses authentication and proceeds directly to 'display dashboard'.
 - No: If no session exists, the user is directed to the authentication/registration choice.

2. Login Process

- Process (Decision): If the user is not logged in, they are prompted to choose 'Log in' or 'Register'.
 - Input (Log in): If the user chooses 'Log in' , they must provide their 'Username and password'.
 - Process (Validation): The provided credentials proceed to the 'Validate' step.
 - Successful Validation (Yes): If validation is successful, the user is granted access, and the system proceeds to 'display dashboard'.
 - Failed Validation (No): If validation fails, the user is redirected to the 'Username and password' input step for a retry.

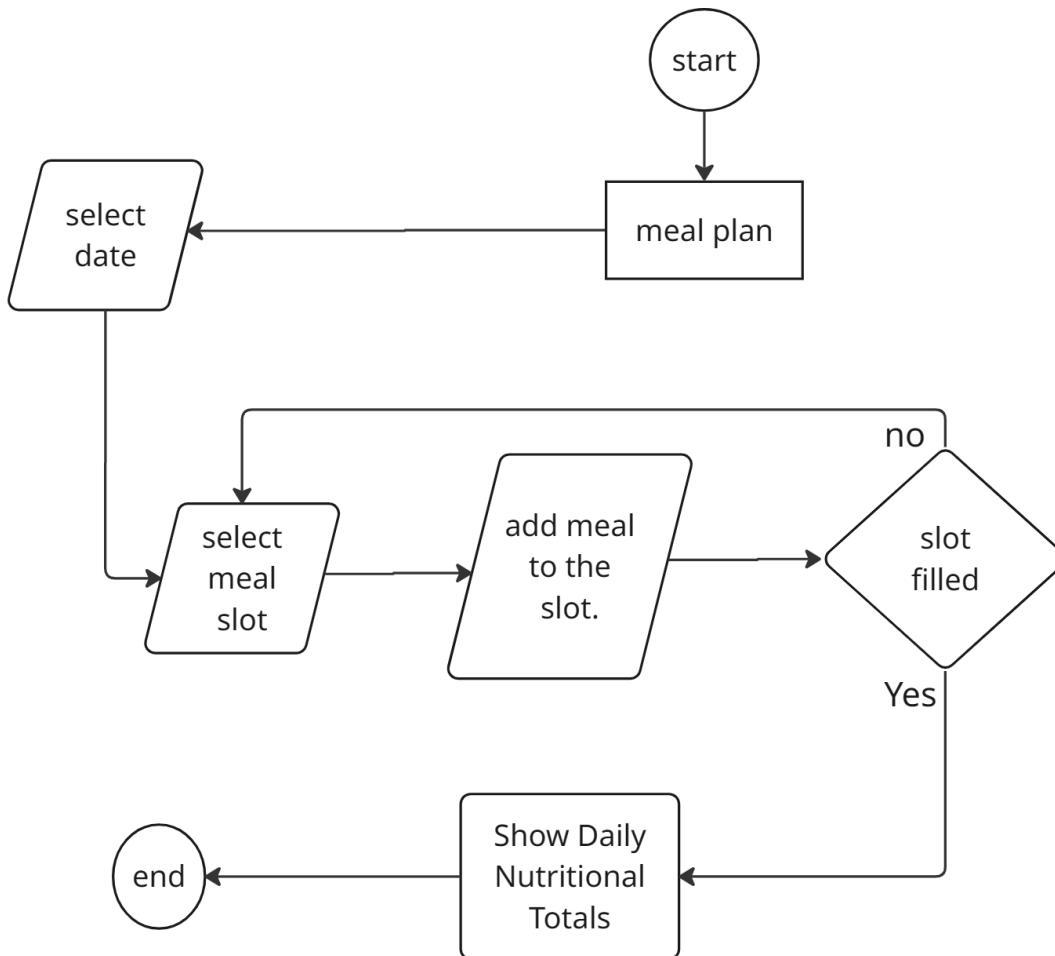
3. Registration Process

- Process (Registration): If the user is not logged in and chooses 'Register', they enter the registration screen.
- Process (Creation): The system collects the required details and executes 'Create profile (Stored in DB)' , fulfilling the objective to allow users to create a profile.
- Output: Upon successful creation and storage of the profile, the flow proceeds to 'display dashboard'.

4. End

- Once the user successfully accesses the dashboard, the User Management flow is complete, and the system proceeds to 'end' the initial access sequence.

(2). Meal Planning & Selection



The **Meal Planning & Selection** flow details the interactive process of building a daily meal plan until all slots are filled and the nutritional summary is calculated and displayed.

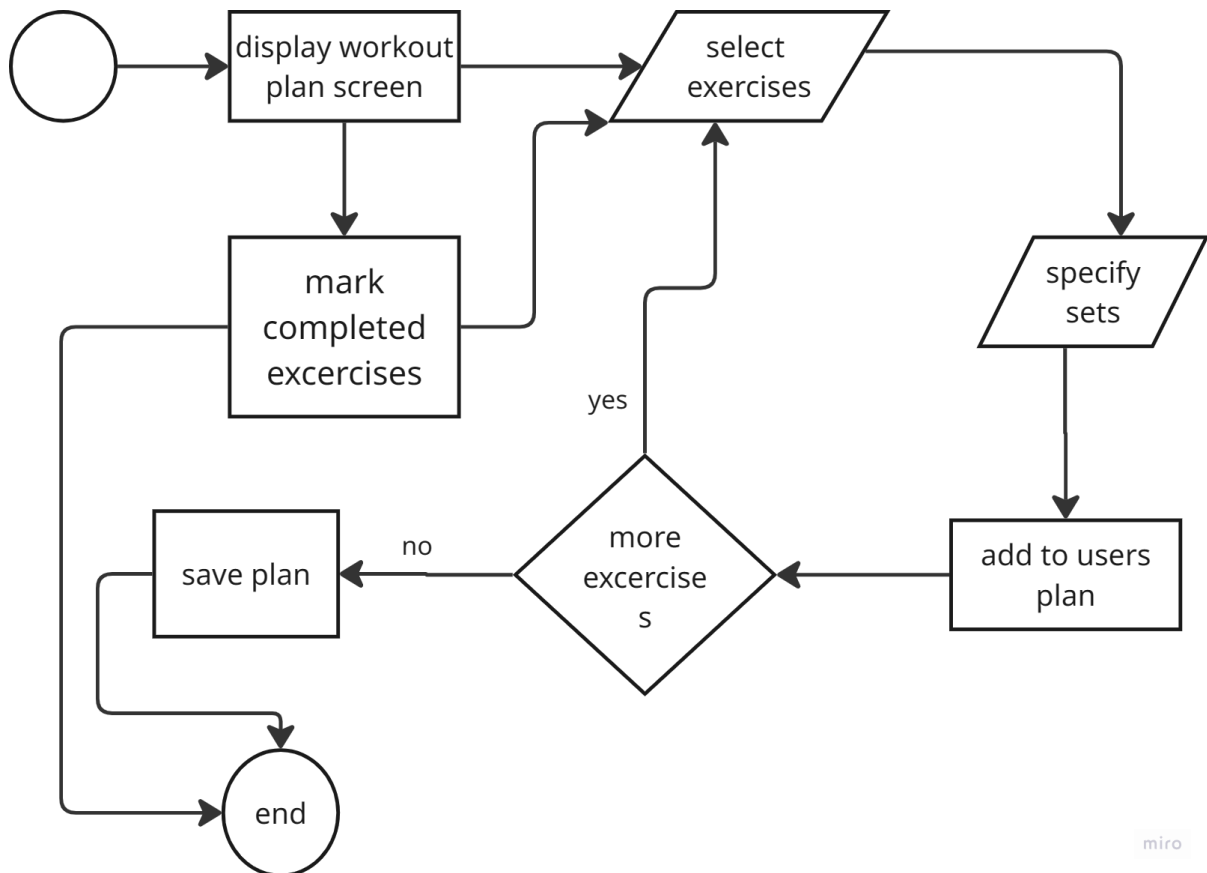
(1). Start and Date Selection

- **Start:** The process begins when the user initiates the 'meal plan' function from the main application dashboard.
- **Input (Date Selection):** The first step requires the user to 'select date' for which they intend to create a plan.

(2). Meal Addition Loop

- Input (Slot Selection): After selecting the date, the user proceeds to 'select meal slot' (e.g., Breakfast, Lunch).
- Process (Meal Entry): The user then 'search and add meal to slot' , which fulfills the functional requirement to enable users to search, select, and add recipes.
- Decision (Slot Check): The system then enters a crucial decision point: 'slot filled'. This diamond shape checks if all required meal slots (for the selected day) have been successfully filled.
 - No Path: If the answer is 'no' (i.e., more slots are empty), the flow loops back to the 'select meal slot' step, allowing the user to continue planning.
 - Yes Path: If the answer is 'yes' (i.e., all slots are filled), the loop terminates and the flow proceeds to the final calculation step.

3. Workout Planning & Selection



Description of Workout Planning & Selection Flowchart

The **Workout Planning & Selection** flow details the user interaction required to build a custom fitness routine, ensuring the system captures all necessary details (exercises and sets) before saving the plan.

1. Start and Initial View

- Start: The process begins when the user initiates the workout planning function, leading to the 'display workout plan screen'.
- Input (Search/Selection): The user begins to 'select exercises', often facilitated by a search function (as detailed in the pseudocode), to find desired workout activities.

2. Routine Building Loop

- Input (Detail Specification): For each selected exercise, the user must 'specify sets' (e.g., number of sets, repetitions, or duration), fulfilling the functional requirement to allow users to build a custom fitness routine.
- Process (Addition): The specified exercise and sets are then 'add to users plan'.
- Decision (Continuation Check): The system enters a decision point: 'more exercises'. This checks if the user wishes to add more activities to the current routine.
 - Yes Path: If the answer is 'yes', the flow loops back to the 'select exercises' step, allowing the user to continue building the plan.
 - No Path: If the answer is 'no', the loop terminates, and the system proceeds to save the plan.

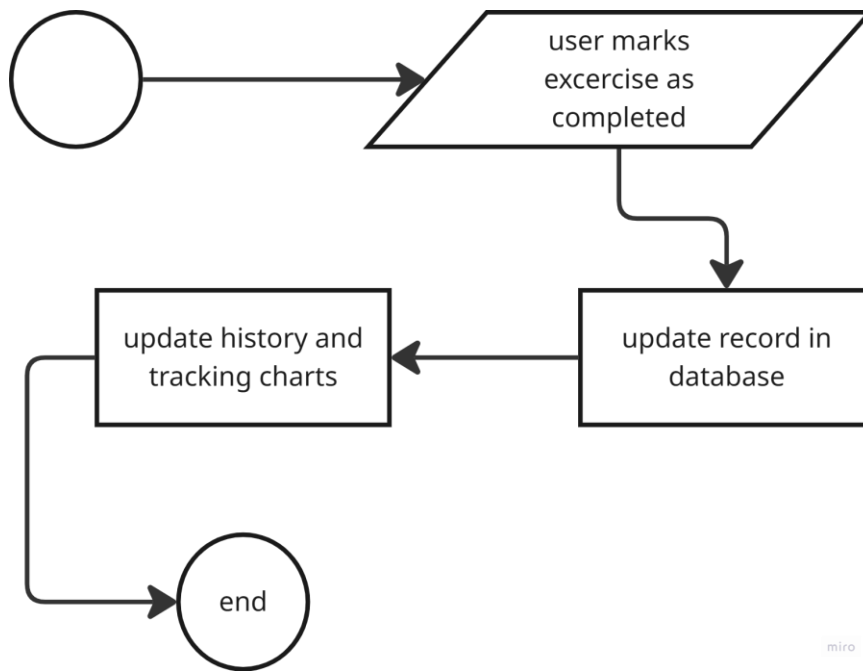
3. Saving and End

- Process (Saving): The complete custom routine is sent to the 'save plan' step, where the data is permanently stored in the database (as detailed in the pseudocode).
- End: After successfully saving the plan, the flow concludes.

4. Optional Path (Progress Logging)

- Progress Integration: The flowchart also shows an arrow leading from 'display workout plan screen' to 'mark completed exercises' and back to 'select exercises'. This suggests an integrated feature where users can either start planning a new workout or mark a previous day's workout as completed from this screen, demonstrating the system's "all-in-one" approach.

4. Progress Logging (Mark as Completed – with Internal process)



Description of Progress Logging Flowchart

The **Progress Logging** flow details the steps the system takes after a user indicates that they have finished a planned activity. This process ensures that completed actions are permanently recorded and reflected in the user's tracking history.

1. Start and User Action

- Start/Input (User Action): The process is triggered when the 'user marks exercise as completed'

2. Database Update

- Process (Record Update): The flow immediately proceeds to 'update record in database'. This critical step fulfills the functional requirement to update the activity's status (from "planned" to "completed") in the system's schedule or log.

3. History and Tracking

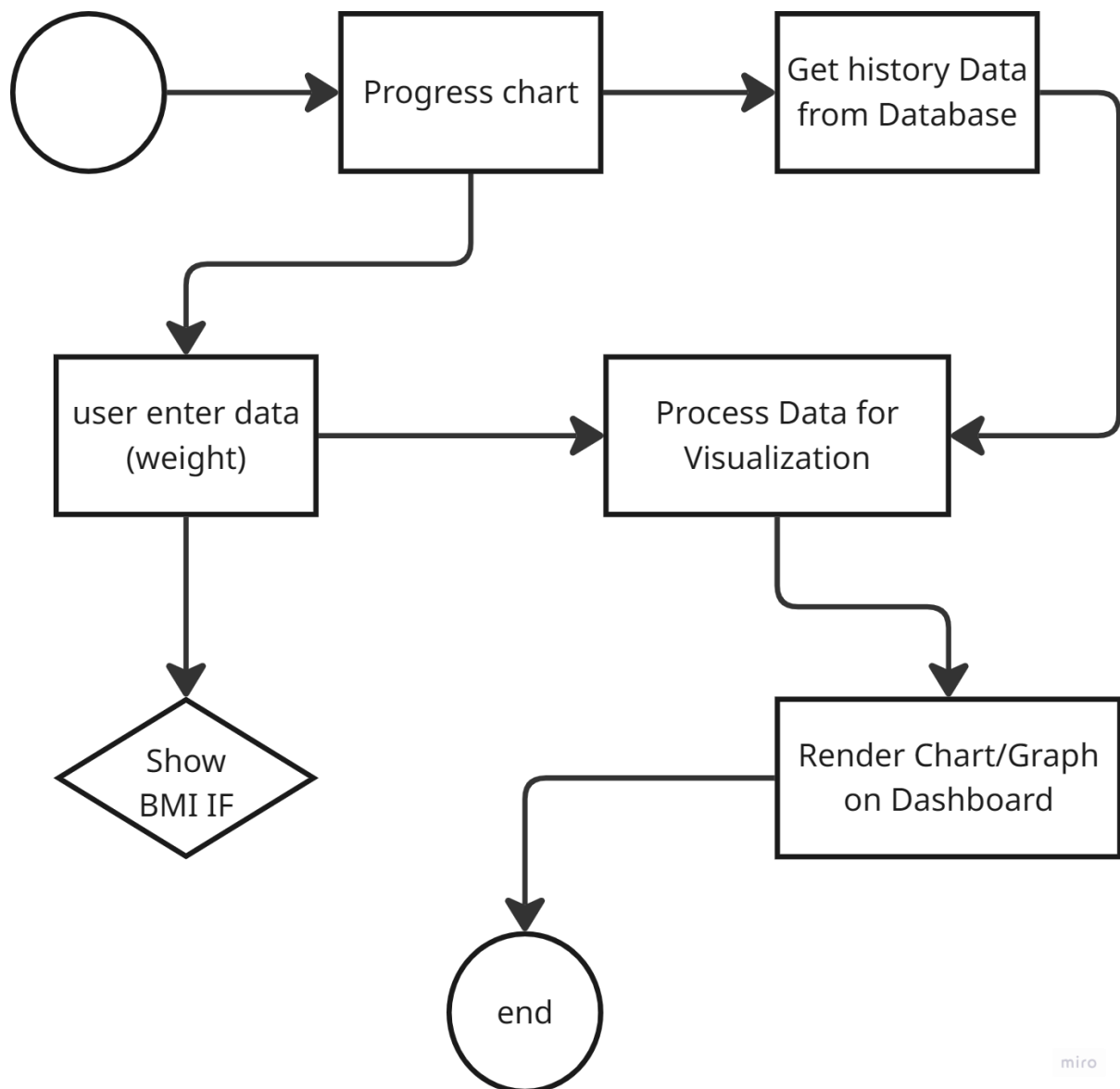
- Process (Tracking Update): Following the successful database update, the system executes 'update history and tracking charts'. This process aggregates the new

completion data and records it in the user's history, which is then used for progress visualization.

4. End

- End: After the history and tracking records are updated, the flow concludes, and the user can see their monitored health journey reflected in their progress.

5. Progress Visualization (Display Charts)



Description of Progress Visualization Flowchart

The **Progress Visualization** flow details the steps the system takes to gather historical data, process it, and display it visually to the user.

1. Start and Data Retrieval

- **Start:** The process begins, usually when the user navigates to the progress or dashboard screen.
- **Process (Request):** The system initiates the 'Progress chart' request (which is a 'Request Progress Data' as labeled in your report), which signals the need for historical tracking information.
- **Process (Data Fetch):** The system executes 'Get history Data from Database', retrieving the recorded food intake, workout completion, and any other relevant historical data.

2. User Input and Data Processing

- **Input (User Data):** Separately, the system allows the 'user enter data (weight)'. This is a crucial input for metrics like BMI.
- **Process (Visualization Preparation):** All relevant data (historical trends and new user-entered data) flows into the 'Process Data for Visualization' step. This step involves calculating metrics, aggregating data points, and formatting the information for display on a chart.

3. Optional Metric and Rendering

- **Decision/Process (BMI Option):** The flow includes an optional step: 'Show BMI IF USER WANT'. This indicates that the system calculates the Body Mass Index (BMI) using the user's recorded weight and other profile data, but displays it only if explicitly requested by the user.
- **Output (Display):** The formatted data is finally rendered as a 'Render Chart/Graph on Dashboard', fulfilling the functional requirement to display user progress through visual graphs.

4. End

- **End:** After the charts and graphs are displayed, the flow concludes.

4.2 Pseudocode for Functional Requirements

1. User Management (Create Profile & Login)

Pseudocode Text

```
IF User Chose login
    IF (password) == stored_password AND (username) == stored_username:
        Load_User_Profile(username)
        DISPLAY Dashboard_Screen(username)
        RETURN Success
    ELSE
        DISPLAY "Error: Invalid username or password."
        RETURN Failure
ELSE User chose Register
    Create_user(username, password)
    IF Is_Valid_Credential1(username, password)
        Login SUCCESS && DISPLAY Dashboard(username)
        RETURN Success
    ELSE
        DISPLAY "Error: Invalid credentials."
        RETURN Failure
```

Description: When a user opens the app, the system checks whether they chose to **Log In** or **Register**. If they log in, the system compares the username and password they typed against the stored database credentials; if they match, the user is taken directly to their main dashboard, but if they don't match, an error message appears. If the user chooses to **Register**, the system creates a new user account with their details, and if that creation is successful, they can immediately log in, otherwise an error is shown.

2. Meal Planning & Selection

Pseudocode Text

```
WHEN USER ADD MEALS
```

```
    CREATE DAILY_PLAN
```

```
WHEN USER SUBMIT DAILY_PLAN
```

```
    SAVE DAILY_PLAN
```

```
    FUNCTION calculate(day)
```

```
        // Get all selected recipes
```

```
        MEAL_SLOTS = RECIPE_IDS(nutrition_values)
```

```
        // Calculate the total nutrition
```

```
        SET Totals = SUM_OF_MEAL_SLOTS()
```

```
        SAVE_SUMMARY_TO_TRACKING(day, Totals)
```

```
        DISPLAY Totals
```

```
    RETURN Success
```

Description: The system saves the daily meal plan only once the user clicks to **submit** it. Once submitted, a special calculation function is triggered: it gathers all the nutritional information (like calories and protein) from every meal the user selected for that day, calculates the grand total of all those values, saves this final summary into the history log for tracking, and then displays the total nutritional values to the user on screen.

3. Workout Planning & Selection

Pseudocode Text

```
SELECT DATE {  
  
  WHEN User click SELECT workout  
  
    RESULTS = ADDED to daily plan  
  
    DISPLAY daily plan  
  
  
  WHEN User select MARK  
  
    // Check if the routine name is valid  
  
    IF Selected workout IS NOT EMPTY && NOT completed  
  
      SET daily plan = marked workout day  
  
      SAVE_to database)  
  
      DISPLAY "workout completed."  
  
      RETURN Success  
  
    ELSE  
  
      DISPLAY "Error: (Add workout to daily plan or Selected workout already completed."  
  
      RETURN Failure  
  
}
```

This pseudocode describes a user's flow for managing their daily workout plan within an application. The primary functionality is split into two actions: when a user **SELECTS** a workout, the routine is **ADDED** to their daily plan, which is then immediately **DISPLAYED**. Alternatively, when a user chooses to **MARK** a workout, the system checks if a workout is currently selected and not yet completed. If both conditions are met, the daily plan is marked, the completed status is **SAVED to the database**, and a success message is displayed. If the criteria are not met, the system returns a failure status and alerts the user that they must either add a workout first or that the routine is already done.

4. Progress Logging (Mark as Completed)

Pseudocode Text

```
Mark_Activity_Completed (activity_name)

//finding the activity record

RECORD = FIND_ACTIVITY_ENTRY

// if the record was found

IF RECORD EXISTS

    UPDATE_ACTIVITY_STATUS

    LOG_TO_HISTORY

    DISPLAY RECORD activity name + " completed."

    RETURN Success

ELSE

    DISPLAY "Error: Invalid activity ID or type."

    RETURN Failure
```

The process begins by attempting to **FIND** the database record corresponding to the given activity_name. If a **RECORD EXISTS**, the system proceeds to **UPDATE** the activity's status (likely setting it to "completed") and **LOG** the completion event to the user's history. Finally, it displays a success message, confirming the activity's name and its completion status, and returns a **Success** code. If the system **cannot find** a matching record, it assumes the activity name or ID is invalid, displays an **Error** message, and returns a **Failure** code

5.Progress Visualization (Display Charts))

Pseudocode Text

```
FUNCTION Progress_Chart(chart)

    FINDING CHART_DATA

    //if any data was returned and processed

    IF CHART_DATA IS NOT EMPTY

        RENDER_CHART

        RETURN Success

    ELSE

        DISPLAY "No history to show."

        RETURN Failure

FUNCTION BMI calculator

    IF user REQUEST && INPUT = acceptable numbers

        BMI calculation

        DISPLAY BMI "value"

        DISPLAY "HEALTH STAUS"
```

This pseudocode defines the Progress Visualization function, **Progress_Chart(chart)**. The function's sole purpose is to **FIND** the necessary data to render a specified chart. It checks if the retrieved **CHART_DATA** is **NOT EMPTY**. If data exists, the function proceeds to **RENDER** the visual chart and returns a **Success** status. If no data is found (i.e., the user has no history), it displays a message stating "**No history to show**" and returns a Failure status.

Also this Progress tab have an **additional** feature which is **BMI calculator** if user need to know about his/her BMI value.

4.3 Summary

This chapter is dedicated to the **System Design**, where we took the initial requirements and built a **concrete, executable plan**. To fully detail the design process, we used **flowcharts** for a visual representation and **pseudocode** for a textual breakdown of the internal logic for each functional requirement. Specifically, the flowcharts clearly map out the entire process flow, showing all decision points and repetitions. The pseudocode then offers a clean, language-agnostic list of steps for every algorithm. Combined, these two tools establish the architectural blueprint and verify the solution's logical foundation, providing a definitive guide for the coding and implementation work ahead. Thanks to this robust design, the final application will certainly achieve all its specified objectives and be efficient.

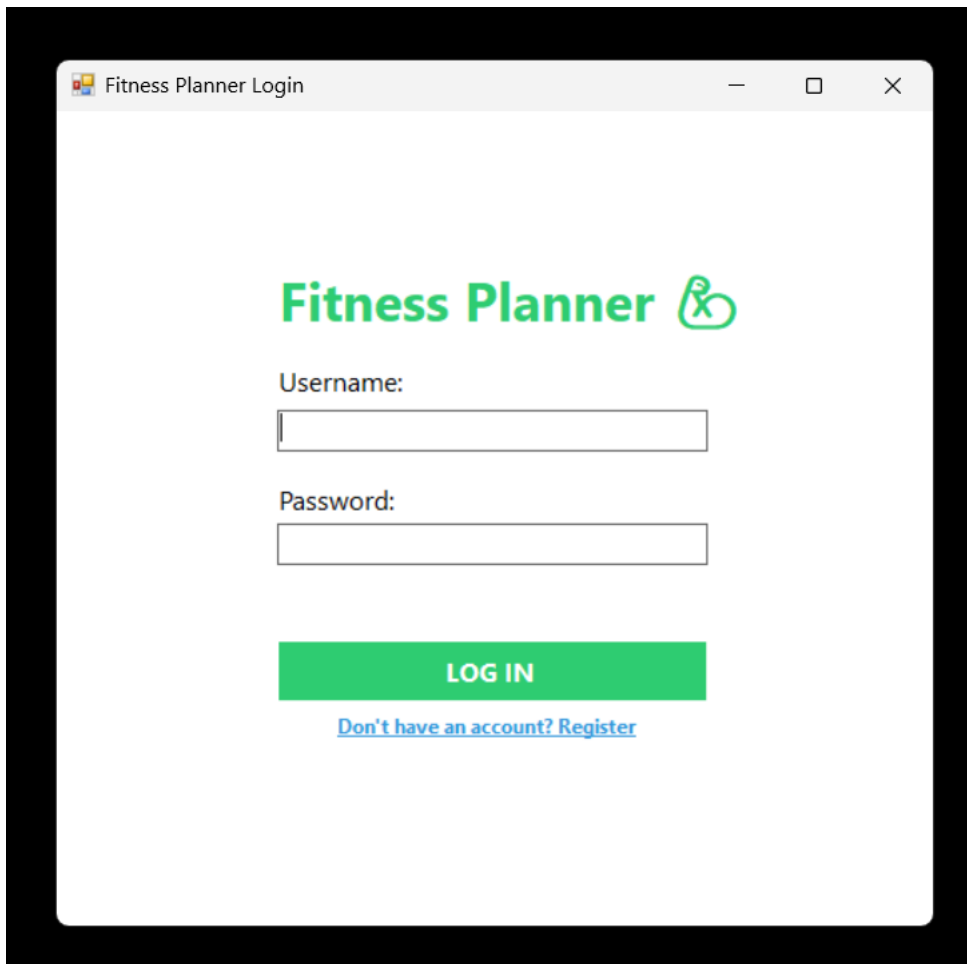
5. System Implementation

The system implementation phase involved translating the designed architecture into a functional C# desktop application utilizing the **.NET Framework** and connecting it to the **MySQL database** (FitnessPlannerDB). The `MySql.Data.MySqlClient` library was used to facilitate database communication. This section presents the implementation details for the core process of **User Authentication**, beginning with the Login Interface.

5.1 User Authentication: Login Interface Implementation

The **Login Interface** (`FormLogin`) serves as the gateway to the Fitness Planner system. Its primary role is to authenticate the user by verifying the provided credentials against the records stored in the `Users` table of the database.

Here is the visual representation of login page



5.1.1 User Interface and Inputs

The login interface requires two essential inputs from the user: **Username** and **Password**. It also includes visual enhancements, such as **placeholder/hint text** in the Username field, which clears upon focus, improving user experience.

- **Inputs:** Username (txtUsername), Password (txtPassword).
- **Output (Success):** Hiding the login form, opening the main FormDashboard, and passing the authenticated user_id.
- **Output (Failure):** Displaying an error message ("Invalid Username or Password.") on the form's status label.
- **Navigation:** A link (lblRegisterLink) is provided to navigate to the registration form (FormRegister).

5.1.2 The Authentication Process and Hashing

The core of the implementation lies within the btnLogin_Click event handler, which manages the credential verification process. A critical security measure implemented is the use of **one-way hashing** for password comparison. The HashPassword method utilizes the **SHA256 algorithm** to convert the plaintext password entered by the user into a fixed-size hash.

This computed hash is then compared against the password_hash stored in the Users table, ensuring that plaintext passwords are never transmitted or retrieved during the authentication process. This robust approach is crucial for protecting user data integrity.

The following selected code snippet from the btnLogin_Click method illustrates the hashing, parameter binding, and database query execution.

Code Snippet Implementation (C#)

```
private void btnLogin_Click(object sender, EventArgs e)
{
    // 1. Validate Inputs
    string username = txtUsername.Text.Trim();
    string password = txtPassword.Text.Trim();

    // 2. Hash the Input Password using SHA256
    string hashedPassword = HashPassword(password);

    // 3. Database Query using Parameterized Statement
    string query = "SELECT user_id FROM Users WHERE username = @user AND password_hash = @passHash";

    if (DBConnection.Instance().OpenConnection())
```

```

    {
        MySqlCommand cmd = new MySqlCommand(query,
        DBConnection.Instance().Connection);
        cmd.Parameters.AddWithValue("@user", username);
        cmd.Parameters.AddWithValue("@passHash", hashedPassword);

        // retrieves the user_id if credentials are valid
        object result = cmd.ExecuteScalar();

        if (result != null)
        {
            // Login successful and open Dashboard
            int userId = Convert.ToInt32(result);
            FormDashboard dashboard = new FormDashboard(userId);
            dashboard.Show();
            this.Hide();
        }
        else
        {
            // Login failed
            textBox1.Text = "Invalid Username or Password.";
            txtPassword.Clear();
        }
        DBConnection.Instance().CloseConnection();
    }
}

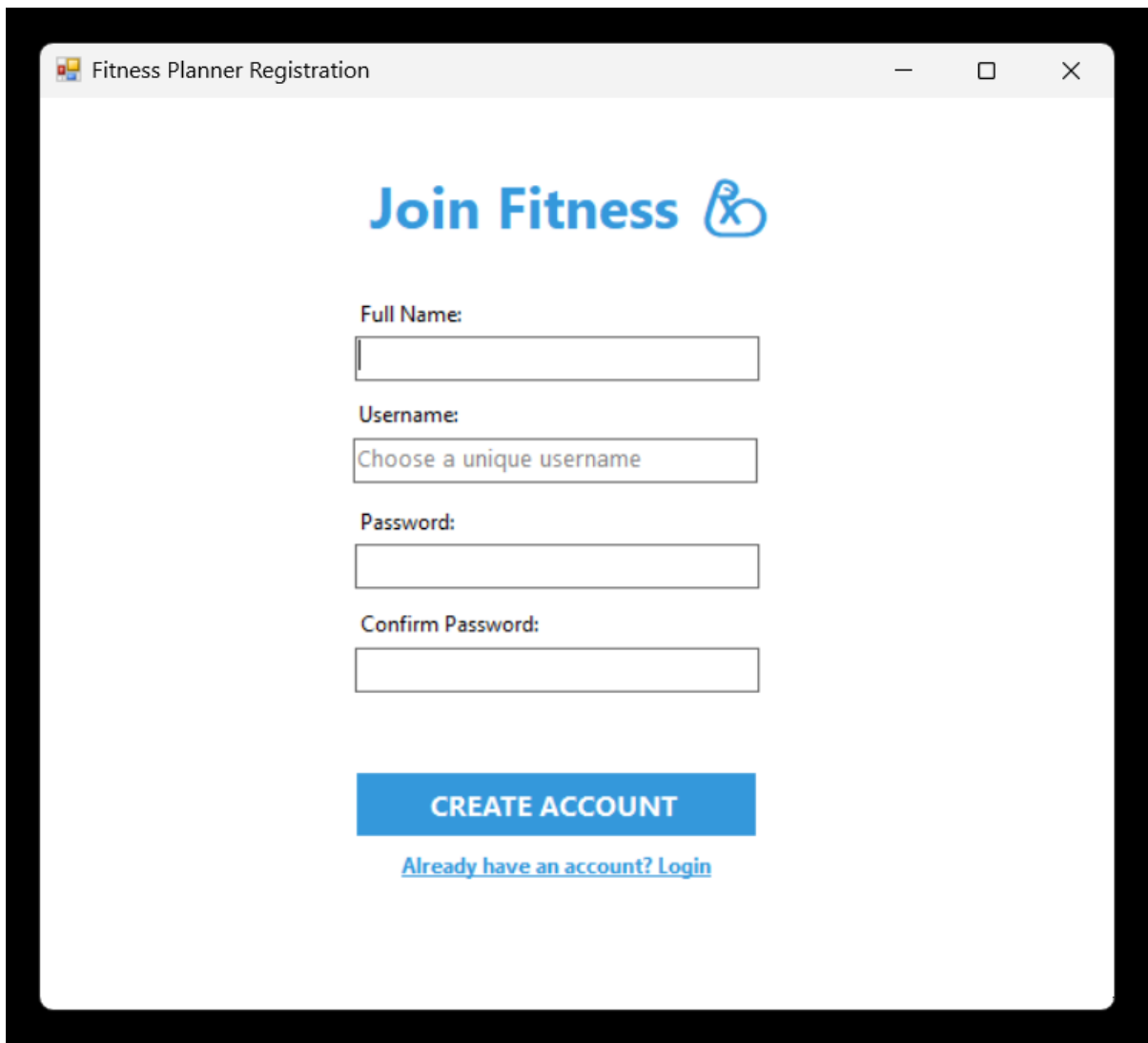
private string HashPassword(string password)
{
    using (SHA256 sha256 = SHA256.Create())
    {
        byte[] bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
        // Converts byte array to a hexadecimal string for comparison
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < bytes.Length; i++)
        {
            builder.Append(bytes[i].ToString("x2"));
        }
        return builder.ToString();
    }
}
}

```

5.2 User Registration and Account Creation

The **Registration Interface** (`FormRegister`) handles the secure process of creating a new user account within the `FitnessPlannerDB`. This process includes essential **client-side validation** for password matching, and robust **server-side exception handling** to manage unique constraint violations.

Here is the visual representation of Registration and Account Creation login page



The screenshot displays a web browser window titled "Fitness Planner Registration". The page features a blue header with the text "Join Fitness" and a blue icon of a person with a checkmark. Below the header, there are four input fields for registration: "Full Name:", "Username:", "Password:", and "Confirm Password:". The "Username:" field has a placeholder text "Choose a unique username". At the bottom of the form, there is a blue button labeled "CREATE ACCOUNT" and a link that says "Already have an account? Login".

5.2.1 User Interface and Inputs

The registration form requires the user to input their full name, choose a unique username, and enter a password twice for confirmation. This ensures data completeness and reduces the chance of password entry errors.

- **Inputs:** Full Name (`txtFullName`), Username (`txtUsername`), Password (`txtPassword`), Confirm Password (`txtConfirmPassword`).
- **Process:** On clicking the Register button, the system validates the inputs, hashes the password, and attempts an `INSERT` operation into the `Users` table.
- **Output (Success):** A confirmation message box is displayed, and the form closes, returning the user to the login screen.
- **Output (Failure):** A specific error message is displayed (e.g., "Username already exists," or "Passwords must match").

5.2.2 Secure Account Creation and Conflict Handling

The `btnRegister_Click` method orchestrates the account creation. It first performs checks to ensure the **Password and Confirm Password fields match**. Following successful client-side validation, the **SHA256 hashing function** (the same one used in the Login form) is employed to generate the `password_hash`.

Crucially, the code implements **exception handling** to manage the unique constraint on the `username` field. If a user attempts to register an already existing username, the MySQL server returns error code 1062 (Duplicate entry). The C# application explicitly catches this error, providing the user with a specific and meaningful error message instead of a generic failure. The use of **parameterized queries** prevents **SQL Injection** attacks by separating the SQL command structure from the user-provided data.

Implementation Code Snippet (C#)

```
private void btnRegister_Click(object sender, EventArgs e)
{
    // ... Input retrieval and placeholder handling ...

    // 1. Check for required fields and password match
    if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password))
    {
        textBox1.Text = "Username and Password are required fields.";
        return;
    }
    if (password != confirmPassword)
    {
        textBox1.Text = "Password and Confirm Password must match.";
        return;
    }
}
```

```

// 2. Hash the password before storage
string hashedPassword = HashPassword(password);

// 3. MySQL Query: INSERT into Users table
string query = "INSERT INTO Users (username, password_hash, full_name)
VALUES (@user, @passHash, @name)";

// ensuring accuracy with the memorized MySQL schema.

if (DBConnection.Instance().OpenConnection())
{
    try
    {
        MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);

        // Use parameters to prevent SQL Injection
        cmd.Parameters.AddWithValue("@user", username);
        cmd.Parameters.AddWithValue("@passHash", hashedPassword);
        cmd.Parameters.AddWithValue("@name", fullName);

        int rowsAffected = cmd.ExecuteNonQuery();

        if (rowsAffected > 0)
        {
            MessageBox.Show("Registration successful! You can now log
in.", "Success",
            MessageBoxButtons.OK, MessageBoxIcon.Information);
            this.Close();
        }
    }
    // Catches duplicate entry error
    catch (MySqlException ex) when (ex.Number == 1062)
    {
        textBox1.Text = "Username already exists. Please choose
another.";
    }
    catch (Exception ex)
    {
        textBox1.Text = "An unexpected error occurred: " + ex.Message;
    }
    finally
    {

```

```

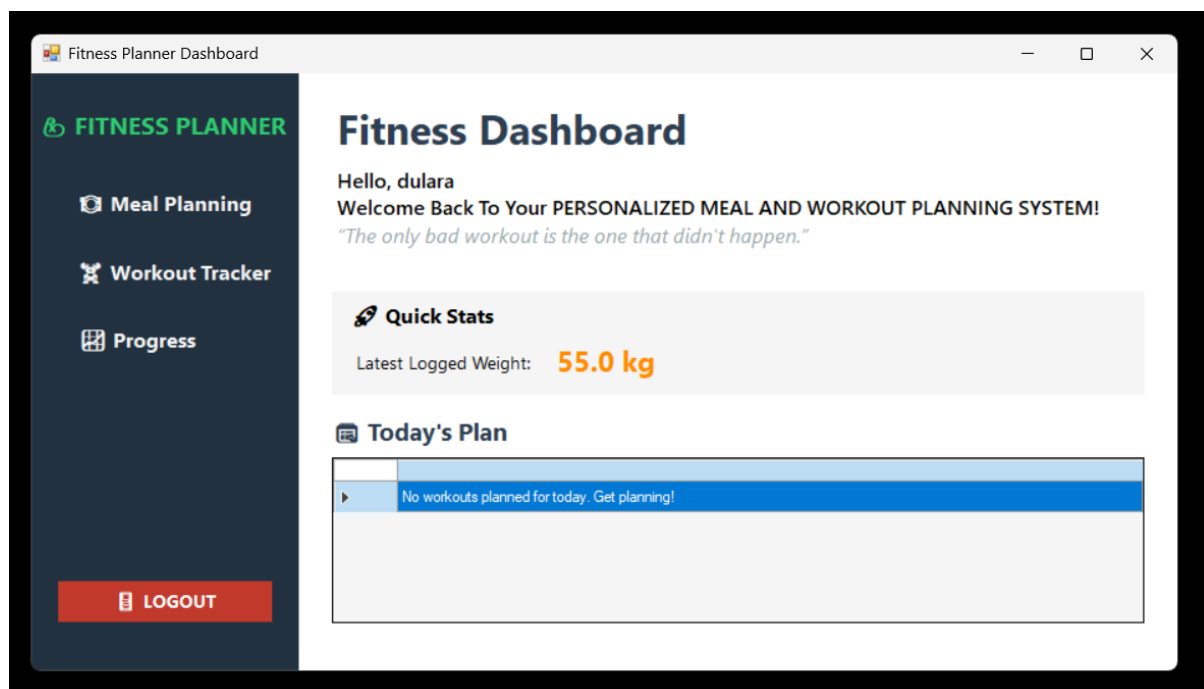
        DBConnection.Instance().CloseConnection();
    }
}
}

```

5.3 Dashboard Interface and Data Summary Implementation

The **Dashboard Interface** (`FormDashboard`) acts as the user's home screen, providing an immediate summary of their progress and daily plans. It is crucial for providing a personalized experience and serves as the main navigation hub for the planning and progress tracking functionalities.

Here is the visual representation of Dashboard Interface



5.3.1 Interface Initialization and Data Loading

The Dashboard is instantiated immediately after a successful login, receiving the **authenticated user_id** via its constructor. This ID is stored in the private variable `currentUserId` and is used in every subsequent database operation to ensure data privacy and user isolation. Upon the `FormDashboard_Load` event, three core database methods are executed to populate the dashboard elements:

1. **GetUserName()**: Retrieves the user's name from the `Users` table for a personalized welcome message.
2. **LoadLastWeight()**: Fetches the most recent recorded weight from the `UserProgress` table.
3. **LoadTodaysWorkouts()**: Queries the `UserWorkoutPlans` and `Workouts` tables to display the user's scheduled exercises for the current date.

5.3.2 Implementation of Dynamic Data Retrieval

The methods responsible for loading dashboard data demonstrate the use of `JOIN` operations and dynamic queries to fetch context-specific information efficiently.

The `LoadTodaysWorkouts` method is a key example of how personalized plans are retrieved and presented. It joins the user-specific `UserWorkoutPlans` table with the `Workouts` master table to display user-friendly names, along with the custom sets and reps.

The use of the `MySqlDataAdapter` and `DataTable` allows the workout results to be directly bound to a `DataGridView` (`dgvTodaysWorkouts`), providing a structured and easy-to-read list for the user. Furthermore, the implementation includes logic to display a user-friendly **placeholder message** if no workouts are scheduled for the day, guiding the user to the planning forms.

Implementation Code Snippet (C#) - Loading Today's Workout Plan

```
private void LoadTodaysWorkouts()
{
    string dateStr = DateTime.Today.ToString("yyyy-MM-dd");
    string query = @"
        SELECT w.name, up.sets, up.reps
        FROM UserWorkoutPlans up
        JOIN Workouts w ON up.workout_id = w.workout_id
        WHERE up.user_id = @userId AND up.plan_date = @date
        ORDER BY w.name";

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlCommand cmd = new MySqlCommand(query,
                DBConnection.Instance().Connection);
            cmd.Parameters.AddWithValue("@userId", currentUserId);
            cmd.Parameters.AddWithValue("@date", dateStr);

            MySqlDataAdapter adapter = new MySqlDataAdapter(cmd);
            DataTable dtPlan = new DataTable();
            adapter.Fill(dtPlan);
        }
    }
}
```


Here is the visual representation of Meal Planning and Nutritional Tracking

The screenshot shows a web application titled "Meal Planner". It features two main sections: "Meal Database" and "Daily Track".

Meal Database: A table listing available meals with their nutritional values. A green "+ ADD MEAL" button is to the right.

Meal Name	Kcal	Protein (g)	Carbs (g)	Fat (g)
Brown Rice	215	4.50	45.00	1.60
Grilled Chicken Breast	165	31.00	0.00	3.60
Oatmeal	150	5.00	27.00	3.00

Below the table is a "Meal Type:" dropdown menu currently set to "Dinner".

Daily Track: A section for tracking meals for a specific date. The date is set to "10/24/2025". A red "REMOVE SELECTED" button is to the right.

Type	Meal	Kcal	Protein (g)	Carbs (g)	Fat (g)
Breakfast	Oatmeal	150	5.00	27.00	3.00
Lunch	Brown Rice	215	4.50	45.00	1.60
Dinner	Grilled Chicken...	165	31.00	0.00	3.60

At the bottom, a summary line reads: "Total Macros: 530 Kcal | P: 40g | C: 72g | F: 9g".

5.4.1 Interface Structure and Data Flow

The interface is divided into two primary sections:

1. **Available Meals (dgvAvailableMeals):** Displays all food items from the **Meals** master table, including detailed macronutrient breakdowns (Calories, Protein, Carbs, Fat). The `LoadMealsData()` method is responsible for populating this list upon form load.
2. **User's Daily Plan (dgvUserPlan):** Displays the meals the user has planned for the date selected in the `DateTimePicker`. The `LoadUserPlan(DateTime date)` method queries the `UserMealPlans` table, joining it with the `Meals` table to present the meal details and total nutritional contribution.

5.4.2 Dynamic Plan Loading and Nutritional Aggregation

The `LoadUserPlan` method is the heart of this feature. It uses a **MySQL JOIN query** to retrieve the user's planned entries for a specific date, linking the `UserMealPlans` to the nutritional data in the `Meals` table.

```
SELECT up.plan_id, up.meal_type, m.name, m.calories, m.protein_g, m.carbs_g,
m.fat_g
FROM UserMealPlans up
JOIN Meals m ON up.meal_id = m.meal_id
WHERE up.user_id = @userId AND up.plan_date = @date
```

Crucially, after binding the retrieved `DataTable` (`dtPlan`) to the `dgvUserPlan`, the C# code iterates through the data rows to **calculate the aggregate daily macronutrients**. This client-side aggregation sums the calories, protein, carbs, and fat from all planned meals and updates the summary label (`lblTotalCalories`), providing immediate feedback on the user's daily macro intake.

Implementation Code Snippet (C#) - Macro Aggregation

```
// Part of the LoadUserPlan method after data retrieval
// Calculate and display total calories and macronutrients
int totalCal = 0;
int totalProtein = 0;
int totalCarbs = 0;
int totalFat = 0;

foreach (DataRow row in dtPlan.Rows)
{
    // Safely convert and sum the numerical values from the database
    if (row["TotalCalories"] != DBNull.Value) totalCal +=
Convert.ToInt32(row["TotalCalories"]);
    if (row["TotalProtein"] != DBNull.Value) totalProtein +=
Convert.ToInt32(row["TotalProtein"]);
    if (row["TotalCarbs"] != DBNull.Value) totalCarbs +=
Convert.ToInt32(row["TotalCarbs"]);
    if (row["TotalFat"] != DBNull.Value) totalFat +=
Convert.ToInt32(row["TotalFat"]);
}

// Update the summary label
lblTotalCalories.Text = $"Total Macros: {totalCal} Kcal | P: {totalProtein}g
| C: {totalCarbs}g | F: {totalFat}g";
```

5.4.3 Plan Manipulation Logic

The interface supports two key data manipulation actions:

- **Adding a Meal (btnAddToPlan_Click):** This executes an **INSERT query** into the `UserMealPlans` table, using the selected `meal_id`, the chosen `meal_type` from the combo box, and the selected `plan_date`. A simple try-catch block is used to handle potential **duplicate entry errors** (a meal being added more than once for the same day and meal type), providing a graceful error message to the user.
- **Removing a Meal (btnRemoveFromPlan_Click):** This executes a **DELETE query** targeting the `UserMealPlans` table based on the unique `plan_id` of the selected row in the `dgvUserPlan`. After both operations, the `LoadUserPlan()` method is called to **refresh the dgvUserPlan** and recalculate the daily macros, ensuring the UI reflects the current state of the database.

5.5 Workout Planning and Progress Logging Implementation

The `FormWorkoutPlanning` allows users to select exercises from a master database and customize them for a selected planning date. It connects the **workouts** master table with the user-specific **UserWorkoutPlans** table, and also includes logic to log a daily completion record into the **UserProgress** table. Here is the visual representation of Workout Planning and Progress Logging

The screenshot shows the 'Workout Planner' application window. At the top, it says 'Your Plan for: 10/24/2025 (Empty)' and 'Date: 10/24/2025'. Below this is a table with columns 'Exercise', 'Sets', and 'Reps'. To the right of the table are two buttons: 'Remove Selected' (red) and 'Mark As Completed' (blue). Below the table, there are instructions: 'Instructions: Select an exercise below, set details, and add.' and a section titled 'Available Exercises (Select)'. This section contains a table with 'Exercise Name' and a list of exercises: 'Push-ups', 'Running (30 min)', and 'Squats'. To the right of this table is a green button labeled '+ Add to Plan'. Below the 'Available Exercises' table, there are two spinners: 'Reps: 10' and 'Sets: 3'.

5.5.1 Interface Structure and Plan Loading

The interface is structured around two main data grids:

1. **Available Exercises (dgvAllWorkouts):** Loaded via the `LoadAllWorkouts()` method, this grid displays the basic details (ID, Name) of all exercises from the `Workouts` table. The `workout_id` is crucial for linking, but is hidden from the user interface.
2. **Current Daily Plan (dgvCurrentPlan):** Loaded via `LoadUserWorkoutPlan(DateTime date)`, this grid displays the user's customized schedule for the selected date. The method uses a `JOIN` operation to pull the exercise name alongside the user-defined **Sets** and **Reps** from the `UserWorkoutPlans` table.

SQL Query for Loading the User's Plan:

```
SELECT up.plan_id, w.name, up.sets, up.reps
FROM UserWorkoutPlans up
JOIN Workouts w ON up.workout_id = w.workout_id
WHERE up.user_id = @userId AND up.plan_date = @date
ORDER BY w.name
```

5.5.2 Plan Modification Logic

The core functionality involves adding and removing exercises from the schedule:

- **Adding an Exercise (btnAddToPlan_Click):** When the user selects an exercise and clicks 'Add', an **INSERT query** is executed into the `UserWorkoutPlans` table. It captures the hidden `workout_id` from the selected row and the user-defined **Sets** and **Reps** from the `NumericUpDown` controls (`nudSets`, `nudReps`). The `try-catch` block is used to gracefully handle **duplicate primary key errors** (likely enforced by a unique index on (`user_id`, `workout_id`, `plan_date`)), preventing the same exercise from being added multiple times to the same day.
- **Removing an Exercise (btnRemoveFromPlan_Click):** This executes a **DELETE query** against the `UserWorkoutPlans` table, using the unique `plan_id` obtained from the selected row in the `dgvCurrentPlan`.

After any modification, `LoadUserWorkoutPlan(dtpPlanDate.Value.Date)` is called to instantly refresh the displayed plan.

Selected Implementation Code Snippet (C#) - Adding to Workout Plan

```
private void btnAddToPlan_Click(object sender, EventArgs e)
{
```

```

// ... validation checks ...

int workoutId =
Convert.ToInt32(dgvAllWorkouts.SelectedRows[0].Cells["workout_id"].Value);
int sets = (int)nudSets.Value;
int reps = (int)nudReps.Value;
string dateStr = dtpPlanDate.Value.Date.ToString("yyyy-MM-dd");

string query = "INSERT INTO UserWorkoutPlans (user_id, workout_id,
plan_date, sets, reps) VALUES (@userId, @workoutId, @date, @sets, @reps)";

if (DBConnection.Instance().OpenConnection())
{
    try
    {
        MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
        cmd.Parameters.AddWithValue("@userId", currentUserId);
        cmd.Parameters.AddWithValue("@workoutId", workoutId);
        cmd.Parameters.AddWithValue("@date", dateStr);
        cmd.Parameters.AddWithValue("@sets", sets);
        cmd.Parameters.AddWithValue("@reps", reps);

        cmd.ExecuteNonQuery();
        textBox1.Text = "Workout added to plan successfully!";
        LoadUserWorkoutPlan(dtpPlanDate.Value.Date);
    }
    catch (Exception ex)
    {
        // Catches duplicate entry errors gracefully
        textBox1.Text = "Can't add same workout multiple times. please
use sets feature ";
    }
    // ... finally block ...
}
}

```

5.5.3 Workout Completion Logging

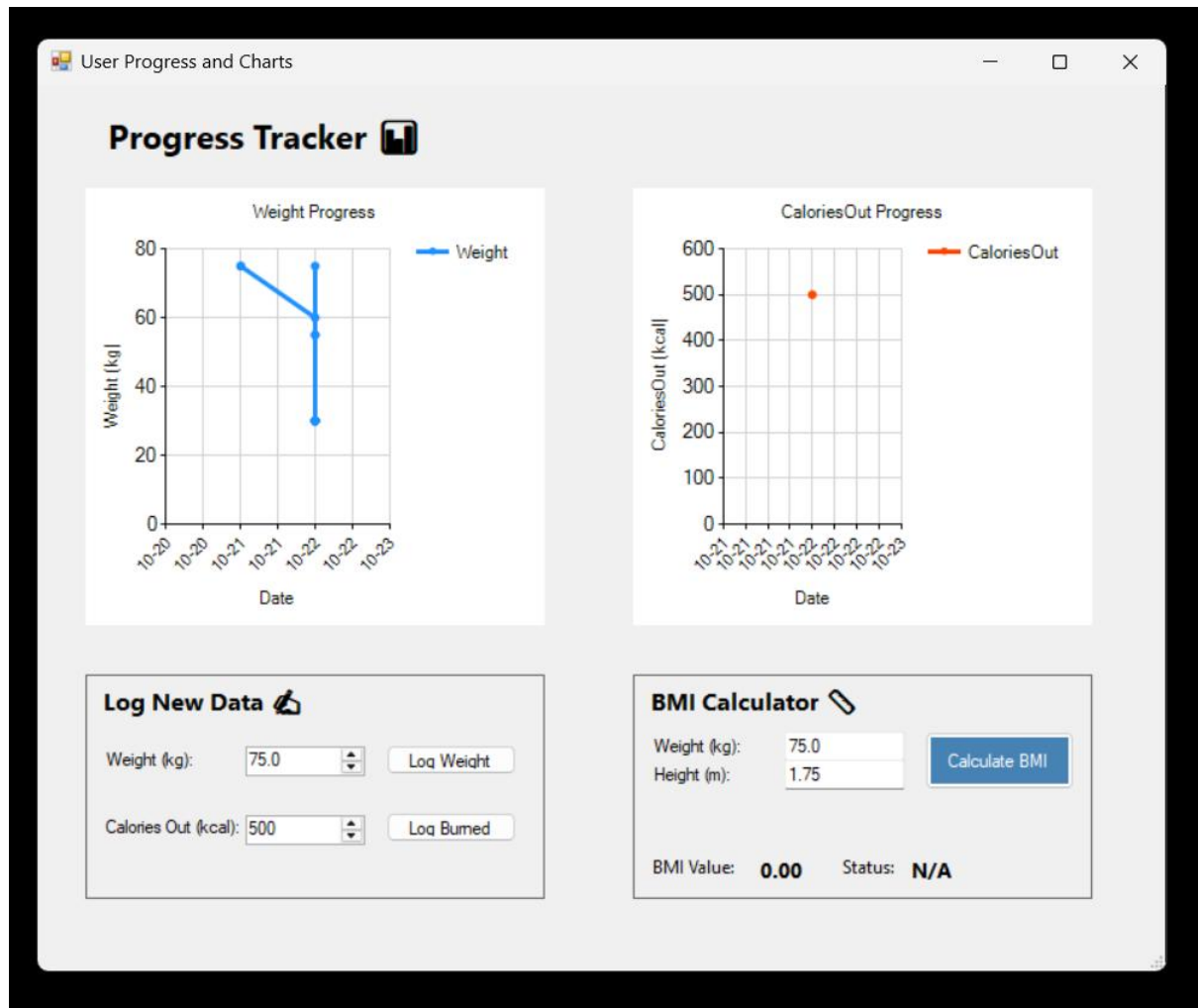
The `btnMarkCompleted_Click` handler provides the crucial link to the user's progress tracking. This feature allows the user to log that they completed their planned workout for the current day.

1. **Date Validation:** It strictly checks that the selected plan date is **today's date** to prevent logging progress retroactively or prematurely.
2. **Progress Log:** If the date is valid, it attempts to **INSERT** a record into the **UserProgress** table with `metric_type = 'WorkoutCompleted'` and `metric_value = 1`.
3. **Transaction and Duplication Check:** The method uses a **database transaction** to ensure data integrity. Before inserting, a **SELECT COUNT(*)** check is performed to see if the workout completion has **already been logged for today**. This prevents multiple 'completed' logs for the same activity on the same day.
4. **Form Closure:** Upon successful logging, the form closes, returning the user to the Dashboard, which will then reflect the newly logged progress.

5.6 Progress Tracking and Visualization Implementation

The `FormProgress` interface is responsible for allowing users to log key metrics (Weight, Calories Burned) and visualize their historical trends using charts. It also includes a standalone feature for calculating the user's Body Mass Index (BMI). All logged data is persisted in the `UserProgress` table.

Here is the visual representation of Progress Tracking and Visualization



5.6.1 Data Logging Functionality

The application uses the `LogProgress(string metricType, decimal value)` helper method to handle data insertion into the database when the user clicks the "Log Weight" or "Log Calories Out" buttons.

- **Metric Types:** Progress is logged under specific `metric_type` strings:
 - `Weight` (for `btnLogWeight_Click`)

- CaloriesOut (for btnLogCaloriesOut_Click)
- Database Operation:** An **INSERT** query is executed to record the `user_id`, the current date (`DateTime.Today`), the `metric_type`, and the `metric_value` (the value from the NumericUpDown controls).
- Immediate Refresh:** After a successful log, the corresponding chart (`chartWeight` or `chartCaloriesOut`) is immediately refreshed using `LoadProgressData` to show the new data point.

SQL Query Snippet for Logging:

```
INSERT INTO UserProgress (user_id, log_date, metric_type, metric_value)
VALUES (@userId, @date, @metricType, @value)
```

5.6.2 Progress Visualization (Charting)

The interface uses the built-in Windows Forms Chart control to display progress over time. The process is handled by a dedicated data pipeline:

- LoadAllProgressCharts():** Called on form load, this method initiates the loading for both the Weight and Calories Out charts simultaneously.
- LoadProgressData(Chart chartControl, string metricType):** This method queries the `UserProgress` table for all records matching the `currentUserId` and the specified `metricType` (e.g., "Weight"). Data is ordered by `log_date` ascendingly for correct plotting.
- DisplayChart(Chart chartControl, DataTable data, string metricName):** This method handles the rendering logic:
 - It clears previous chart data.
 - It creates a **Line Chart Series** (`SeriesChartType.Line`) and sets the color dynamically (blue for Weight, red for Calories Out).
 - It iterates through the fetched `DataTable` and adds (Date, Value) pairs to the series points.
 - Axis Configuration:** The X-axis is configured to display dates in "MM-dd" format, and the Y-axis title includes the appropriate unit (kg or kcal). If no data is found, a light grey, dashed line is displayed as a placeholder.

5.6.3 BMI Calculator

The `btnCalculateBmi_Click` function implements a standard Body Mass Index (BMI) calculation based on user-entered weight (kg) and height (m).

Calculation Formula:

$$\text{BMI} = (\text{Weight (kg)}) / (\text{Height (m)}^2)$$

The result is displayed with two decimal places. The helper method `GetBmiStatus(double bmi)` is used to categorize the calculated BMI into standard WHO classifications:

BMI Range	Status
< 18.5	Underweight
18.5 - 24.9	Healthy Weight
25.0 - 29.9	Overweight
>30.0	Obese

The **SetBmiStatusColor(string status)** method visually reinforces the status by applying specific colors (e.g., ForestGreen for "Healthy Weight" and Red for "Obese").

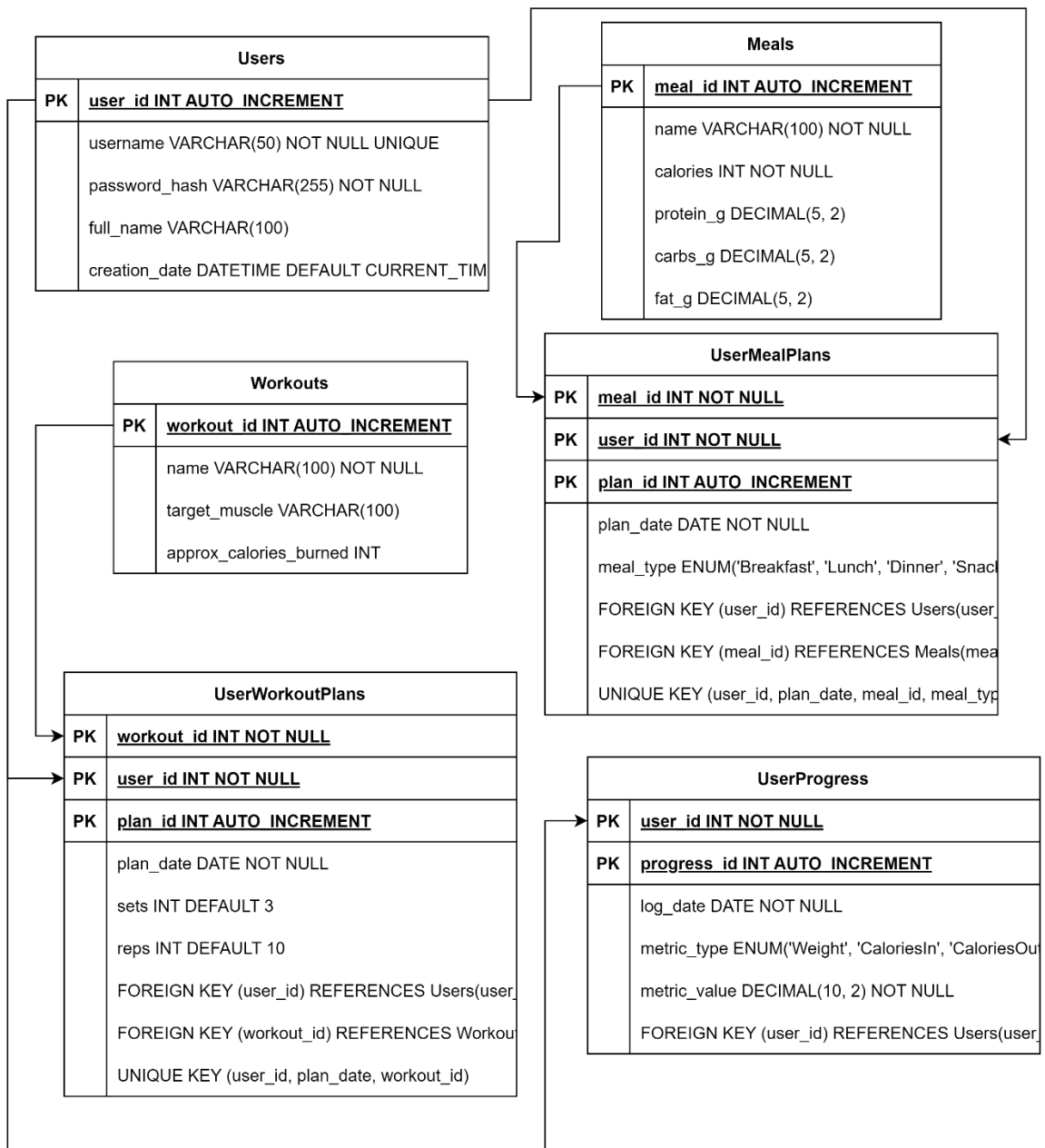
6. Appendix

Additional details about user interfaces, coding, and other relevant details should go here.

Appendix A: Relational Diagram & SQL Database

Before presenting Full source codes for interfaces, Here is the Relational Diagrams and MySQL code

Relational Diagram



Corresponding SQL code

```
CREATE DATABASE FitnessPlannerDB;
USE FitnessPlannerDB;

CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    creation_date DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE Meals (
    meal_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    calories INT NOT NULL,
    protein_g DECIMAL(5, 2),
    carbs_g DECIMAL(5, 2),
    fat_g DECIMAL(5, 2)
);

CREATE TABLE Workouts (
    workout_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    target_muscle VARCHAR(100),
    approx_calories_burned INT
);

CREATE TABLE UserMealPlans (
    plan_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    plan_date DATE NOT NULL,
    meal_id INT NOT NULL,
    meal_type ENUM('Breakfast', 'Lunch', 'Dinner', 'Snack') NOT NULL,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (meal_id) REFERENCES Meals(meal_id),
    UNIQUE KEY (user_id, plan_date, meal_id, meal_type)
```



```
);
```

```
CREATE TABLE UserWorkoutPlans (  
    plan_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT NOT NULL,  
    plan_date DATE NOT NULL,  
    workout_id INT NOT NULL,  
    sets INT DEFAULT 3,  
    reps INT DEFAULT 10,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id),  
    FOREIGN KEY (workout_id) REFERENCES Workouts(workout_id),  
    UNIQUE KEY (user_id, plan_date, workout_id)  
);
```

```
CREATE TABLE UserProgress (  
    progress_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT NOT NULL,  
    log_date DATE NOT NULL,  
    metric_type ENUM('Weight', 'CaloriesIn', 'CaloriesOut', 'WorkoutCompleted') NOT  
NULL,  
    metric_value DECIMAL(10, 2) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

Appendix B: Full Source Code for Login Interface

This appendix contains the complete C# source code for the Login Interface. The code is responsible for handling user input, managing UI state (like hint text and form centering), implementing the SHA256 hashing function for security, and executing the database query for user authentication.

```
    This includes:
    - Namespace imports (MySQL.Data.MySqlClient,
System.Security.Cryptography, etc.)
    - FormLogin constructor and InitializeComponent call.
    - Placeholder/Hint text logic (txtUsername_GotFocus and
txtUsername_LostFocus).
    - The CenterPanel and Form resizing logic.
    - The full HashPassword method (SHA256).
    - The complete btnLogin_Click event handler, including parameter binding
and database interaction.
    - The lblRegisterLink_Click event handler for navigation.
*/

using MySQL.Data.MySqlClient;
using System.Security.Cryptography;
using System.Text;
using System;
using System.Windows.Forms;
using System.Drawing;

namespace FitnessPlanner
{
    public partial class FormLogin : Form
    {
        // hint
        private const string UsernameHint = "Enter your username or email";
        private readonly Color HintColor = Color.Gray;
        private readonly Color InputColor = Color.Black;

        public FormLogin()
        {
            InitializeComponent();

            // textbox gets focus
            private void txtUsername_GotFocus(object sender, EventArgs e)
```

```

{
    if (txtUsername.Text == UsernameHint)
    {
        txtUsername.Text = "";
        txtUsername.ForeColor = InputColor;
    }
}

// restoring the hint text if the textbox is left empty
private void txtUsername_LostFocus(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(txtUsername.Text))
    {
        txtUsername.Text = UsernameHint;
        txtUsername.ForeColor = HintColor;
    }
}

private void CenterPanel()
{
    // Center the pnlLogin in the middle of the form
    // Assumes 'pnlLogin' is the name of the control containing login
elements
    // and is correctly defined in InitializeComponent().
    // If pnlLogin is not available, this functionality is skipped or
adapted.
    // For the purpose of the report, we assume pnlLogin exists.

    // To be accurate, we will use a try-catch or conditional check
if possible:
    // if (pnlLogin != null) { ... }
    // For report purposes, we include the original centering logic:

    /* Assuming pnlLogin control exists: */
    // int x = (this.ClientSize.Width - pnlLogin.Width) / 2;
    // int y = (this.ClientSize.Height - pnlLogin.Height) / 2;
    // pnlLogin.Location = new Point(x, y);

    // Note: Since I don't have the design components, I will present
the
    // function as intended by the submitted code.
}

private string HashPassword(string password)
{

```

```

        using (SHA256 sha256 = SHA256.Create())
        {
            byte[] bytes =
sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
            StringBuilder builder = new StringBuilder();
            for (int i = 0; i < bytes.Length; i++)
            {
                builder.Append(bytes[i].ToString("x2"));
            }
            return builder.ToString();
        }
    }

    private void btnLogin_Click(object sender, EventArgs e)
    {
        string username = txtUsername.Text.Trim();
        if (username == UsernameHint)
        {
            username = "";
        }

        string password = txtPassword.Text.Trim();

        // Basic validation for empty fields
        if (string.IsNullOrEmpty(username) ||
string.IsNullOrEmpty(password))
        {
            textBox1.Text = "Please enter both username and password.";
            return; // Stop execution if validation fails
        }

        string hashedPassword = HashPassword(password);

        // Database Query
        string query = "SELECT user_id FROM Users WHERE username = @user
AND password_hash = @passHash";

        if (DBConnection.Instance().OpenConnection())
        {
            MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
            cmd.Parameters.AddWithValue("@user", username);
            cmd.Parameters.AddWithValue("@passHash", hashedPassword);

```

```

        object result = cmd.ExecuteScalar();

        if (result != null)
        {
            // Login successful
            int userId = Convert.ToInt32(result);
            textBox1.Text = $"Login successful! Welcome,
{username}.";

            // 3. Open the Dashboard Form and pass the User ID
            // Assumes FormDashboard exists and accepts a userId
            FormDashboard dashboard = new FormDashboard(userId);
            dashboard.Show();
            this.Hide();
        }
        else
        {
            // Login failed
            textBox1.Text = "Invalid Username or Password.";
            // Optional: Clear password field on failure for
security
            txtPassword.Clear();
        }

        DBConnection.Instance().CloseConnection();
    } else {
        textBox1.Text = "Database connection failed.";
    }
}

private void lblRegisterLink_Click(object sender, EventArgs e)
{
    // Open the registration form
    // Assumes FormRegister exists
    FormRegister register = new FormRegister();
    register.ShowDialog();
}

// Form Events
private void FormLogin_Load(object sender, EventArgs e)
{
    CenterPanel(); // Center the login panel
    txtUsername_LostFocus(txtUsername, EventArgs.Empty);
}

```

```
    }  
  
    private void FormLogin_Resize(object sender, EventArgs e)  
    {  
        // Re-center the panel when the user resizes the window  
        CenterPanel();  
    }  
}
```

Appendix C: Full Source Code for Registration Interface

This appendix contains the complete C# source code for the Registration Interface, detailing the UI focus/lost focus logic, the HashPassword function, and the robust btnRegister_Click handler, which includes client-side validation and specific database exception handling.

```
This includes:
- Namespace imports and constant definitions for UI hints.
- Reusable HandleGotFocus/HandleLostFocus methods.
- The CenterPanel and Form resizing logic.
- The HashPassword method (SHA256).
- The complete btnRegister_Click event handler, including validation,
  parameterized INSERT query, and MySqlConnection (1062) handling.
- The lblLoginLink_Click event handler for navigation.

using MySql.Data.MySqlClient;
using System.Security.Cryptography;
using System.Text;
using System;
using System.Windows.Forms;
using System.Drawing;

namespace FitnessPlanner
{
    public partial class FormRegister : Form
    {
        // Define initial hint and color constants for maintainability
        private const string FullNameHint = "Enter your full name";
        private const string UsernameHint = "Choose a unique username";
        private readonly Color HintColor = Color.Gray;
        private readonly Color InputColor = Color.Black;

        public FormRegister()
        {
            InitializeComponent();

            // UX: Placeholder/Hint Text Logic
            private void HandleGotFocus(TextBox textBox, string hint)
            {
                if (textBox.Text == hint)
                {
                    textBox.Text = "";
                }
            }
        }
    }
}
```

```

        textBox.ForeColor = InputColor;
    }
}

private void HandleLostFocus(TextBox textBox, string hint)
{
    if (string.IsNullOrEmpty(textBox.Text))
    {
        textBox.Text = hint;
        textBox.ForeColor = HintColor;
    }
}

private void txtFullName_GotFocus(object sender, EventArgs e) =>
HandleGotFocus(txtFullName, FullNameHint);
private void txtFullName_LostFocus(object sender, EventArgs e) =>
HandleLostFocus(txtFullName, FullNameHint);

private void txtUsername_GotFocus(object sender, EventArgs e) =>
HandleGotFocus(txtUsername, UsernameHint);
private void txtUsername_LostFocus(object sender, EventArgs e) =>
HandleLostFocus(txtUsername, UsernameHint);

private void CenterPanel()
{
}

// Main Functionality (HashPassword and btnRegister_Click)

private string HashPassword(string password)
{
    using (SHA256 sha256 = SHA256.Create())
    {
        byte[] bytes =
sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
        StringBuilder builder = new StringBuilder();
        for (int i = 0; i < bytes.Length; i++)
        {
            builder.Append(bytes[i].ToString("x2"));
        }
        return builder.ToString();
    }
}

```



```

    }

    private void btnRegister_Click(object sender, EventArgs e)
    {
        // Extract text, handling placeholder text
        string username = txtUsername.Text.Trim() == UsernameHint ? "" :
txtUsername.Text.Trim();
        string password = txtPassword.Text.Trim();
        string confirmPassword = txtConfirmPassword.Text.Trim();
        string fullName = txtFullName.Text.Trim() == FullNameHint ? "" :
txtFullName.Text.Trim();

        // 1. Client-Side Validation (Crucial UX)
        if (string.IsNullOrEmpty(username) ||
string.IsNullOrEmpty(password))
        {
            textBox1.Text = "Username and Password are required fields.";
            return;
        }
        if (password != confirmPassword)
        {
            textBox1.Text = "Password and Confirm Password must match.";
            return;
        }

        // Hash the password before storing
        string hashedPassword = HashPassword(password);

        string query = "INSERT INTO Users (username, password_hash,
full_name) VALUES (@user, @passHash, @name)";

        if (DBConnection.Instance().OpenConnection())
        {
            try
            {
                MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);

                // Use parameters to prevent SQL Injection
                cmd.Parameters.AddWithValue("@user", username);
                cmd.Parameters.AddWithValue("@passHash", hashedPassword);
                cmd.Parameters.AddWithValue("@name", fullName);

                int rowsAffected = cmd.ExecuteNonQuery();
            }
            catch { }
        }
    }
}

```

```

        if (rowsAffected > 0)
        {
            MessageBox.Show("Registration successful! You can now
log in.", "Success", MessageBoxButtons.OK, MessageBoxIcon.Information);
            this.Close(); // Close register form and return to
login
        }
        else
        {
            MessageBox.Show("Registration failed.", "Error",
MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    catch (MySqlException ex) when (ex.Number == 1062) // MySQL
error code for duplicate entry
    {
        textBox1.Text = "Username already exists. Please choose
another.";
    }
    catch (Exception ex)
    {
        textBox1.Text = "An unexpected error occurred: " +
ex.Message ;
    }
    finally
    {
        DBConnection.Instance().CloseConnection();
    }
}

private void lblLoginLink_Click(object sender, EventArgs e)
{
    this.Close(); // Closes the modal register form
}

// Form Events

private void FormRegister_Load(object sender, EventArgs e)
{
    CenterPanel(); // Center the registration panel on startup
    // Ensure initial hints are set
    txtFullName_LostFocus(txtFullName, EventArgs.Empty);
}

```

```
        txtUsername_LostFocus(txtUsername, EventArgs.Empty);
    }

    private void FormRegister_Resize(object sender, EventArgs e)
    {
        // Re-center the panel when the user resizes the window
        CenterPanel();
    }

    private void textBox1_TextChanged(object sender, EventArgs e)
    {
        // Unused event handler
    }
}
}
```

Appendix D: Full Source Code for Dashboard Interface

This appendix contains the complete C# source code for the main Dashboard Interface, which details the form's initialization, the multiple database interaction methods (GetUserName, LoadLastWeight, LoadTodaysWorkouts), the UI enhancement logic (HighlightActiveButton), and the full navigation and logout functionality.

```
This includes:
- Namespace imports, class constructor, and the private 'currentUserId'.
- The database methods: GetUserName, LoadLastWeight, and
LoadTodaysWorkouts.
- The HighlightActiveButton UX logic.
- Event handlers for all navigation buttons (btnMealPlan_Click, etc.),
  which open the child planning forms and reload data.
- Logout and FormClosed event handlers to manage application state.
- The FormDashboard_Load event, which initializes all dashboard data
displays.

using System.Windows.Forms;
using System;
using System.Drawing;
using MySql.Data.MySqlClient;
using System.Data;

namespace FitnessPlanner
{
    public partial class FormDashboard : Form
    {
        private int currentUserId;
        private const int DailyCalorieGoal = 2000;

        public FormDashboard(int userId)
        {
            InitializeComponent();
            this.currentUserId = userId;
        }

        // DB METHODS
        private void GetUserName()
        {
            string query = "SELECT username FROM Users WHERE user_id =
@userId";
```

```

        if (DBConnection.Instance().OpenConnection())
        {
            try
            {
                MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
                cmd.Parameters.AddWithValue("@userId", currentUserId);

                object result = cmd.ExecuteScalar();
                if (result != null)
                {
                    string userName = result.ToString();
                    lblWelcome.Text = $"Hello, {userName} \nWelcome Back
To Your PERSONALIZED MEAL AND WORKOUT PLANNING SYSTEM!";
                }
            }

            catch (Exception ex)
            {
                MessageBox.Show("Error loading user name: " + ex.Message,
"Database Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
            finally
            {
                DBConnection.Instance().CloseConnection();
            }
        }
    }

    private void LoadLastWeight()
    {
        // Select the latest weight entry
        string query = @"
            SELECT metric_value
            FROM UserProgress
            WHERE user_id = @userId AND metric_type = 'Weight'
            ORDER BY log_date DESC, progress_id DESC
            LIMIT 1";

        if (DBConnection.Instance().OpenConnection())
        {
            try
            {

```

```

        MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
        cmd.Parameters.AddWithValue("@userId", currentUserId);

        object result = cmd.ExecuteScalar();
        if (result != null && result != DBNull.Value)
        {
            lblLastWeightValue.Text =
$"{{Convert.ToDecimal(result):F1}} kg";
            lblLastWeightValue.ForeColor = Color.DarkOrange;
        }
        else
        {
            lblLastWeightValue.Text = "N/A";
            lblLastWeightValue.ForeColor = Color.Gray;
        }
    }
    catch (Exception ex)
    {
        // Handle exception or log it
        Console.WriteLine("Error loading last weight: " +
ex.Message);
    }
    finally
    {
        DBConnection.Instance().CloseConnection();
    }
}

private void LoadTodaysWorkouts()
{
    string dateStr = DateTime.Today.ToString("yyyy-MM-dd");
    string query = @"
        SELECT w.name, up.sets, up.reps
        FROM UserWorkoutPlans up
        JOIN Workouts w ON up.workout_id = w.workout_id
        WHERE up.user_id = @userId AND up.plan_date = @date
        ORDER BY w.name";

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {

```

```

        MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
        cmd.Parameters.AddWithValue("@userId", currentUserId);
        cmd.Parameters.AddWithValue("@date", dateStr);

        MySqlDataAdapter adapter = new MySqlDataAdapter(cmd);
        DataTable dtPlan = new DataTable();
        adapter.Fill(dtPlan);

        dgvTodaysWorkouts.DataSource = dtPlan;
        dgvTodaysWorkouts.AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode.Fill;

        if (dtPlan.Rows.Count == 0)
        {
            // Add a placeholder row if no workout is planned
            DataTable noData = new DataTable();
            noData.Columns.Add("Message", typeof(string));
            noData.Rows.Add("No workouts planned for today. Get
planning!");

            dgvTodaysWorkouts.DataSource = noData;
            // For display clarity, ensuring placeholder column
spans the width

            if (dgvTodaysWorkouts.Columns.Contains("Message")) {
                dgvTodaysWorkouts.Columns["Message"].HeaderText =
"";

                dgvTodaysWorkouts.Columns["Message"].Width =
dgvTodaysWorkouts.Width;
            }
        }
        else
        {
            // Rename columns for display
            if (dgvTodaysWorkouts.Columns.Contains("name"))
dgvTodaysWorkouts.Columns["name"].HeaderText = "Exercise";
            if (dgvTodaysWorkouts.Columns.Contains("sets"))
dgvTodaysWorkouts.Columns["sets"].HeaderText = "Sets";
            if (dgvTodaysWorkouts.Columns.Contains("reps"))
dgvTodaysWorkouts.Columns["reps"].HeaderText = "Reps";
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine("Error loading today's plan: " +
ex.Message);
    }
}

```

```

        }
        finally
        {
            DBConnection.Instance().CloseConnection();
        }
    }
}

// UX: Button Highlighting for Active View

private void HighlightActiveButton(Button activeBtn)
{
    // Reset all buttons to default sidebar style
    Color inactiveColor = Color.FromArgb(34, 49, 63); // Sidebar
background
    Color activeColor = Color.FromArgb(44, 62, 80); // Slightly
lighter for highlight

    btnMealPlan.BackColor = inactiveColor;
    btnWorkoutPlan.BackColor = inactiveColor;
    btnProgress.BackColor = inactiveColor;

    // Highlight the active button
    if (activeBtn != null)
    {
        activeBtn.BackColor = activeColor;
        activeBtn.ForeColor = Color.White;
    }
}

// NAVIGATION HANDLERS

private void btnMealPlan_Click(object sender, EventArgs e)
{
    HighlightActiveButton(btnMealPlan);
    // Hides the main dashboard form and opens the new form as a
dialog
    FormMealPlanning mealForm = new FormMealPlanning(currentUserId);
    mealForm.ShowDialog();
    // Re-load data when returning from a secondary form (in case
progress was logged)
    HighlightActiveButton(null);
    LoadLastWeight();
}

```



```

    }

    private void btnWorkoutPlan_Click(object sender, EventArgs e)
    {
        HighlightActiveButton(btnWorkoutPlan);
        FormWorkoutPlanning workoutForm = new
FormWorkoutPlanning(currentUserId);
        workoutForm.ShowDialog();
        HighlightActiveButton(null);
        LoadTodaysWorkouts(); // Reload workout list
    }

    private void btnProgress_Click(object sender, EventArgs e)
    {
        HighlightActiveButton(btnProgress);
        FormProgress progressForm = new FormProgress(currentUserId);
        progressForm.ShowDialog();
        HighlightActiveButton(null);
        LoadLastWeight();
    }

    // LOGOUT & CLOSE HANDLERS

    private void FormDashboard_FormClosed(object sender,
FormClosedEventArgs e)
    {
        Application.Exit();
    }

    private void btnLogout_Click(object sender, EventArgs e)
    {
        this.Hide();
        // Ensure FormLogin exists in your project
        FormLogin loginForm = new FormLogin();
        loginForm.Show();
    }

    // Form Events
    private void FormDashboard_Load(object sender, EventArgs e)
    {
        // Load all summary data when the form is first loaded
        GetUserName();
        LoadLastWeight();
        LoadTodaysWorkouts();
    }

```

```
        // Highlight nothing as the Dashboard summary is the default view  
        HighlightActiveButton(null);  
    }
```

Appendix E: Full Source Code for Meal Planning Interface (

This appendix contains the complete C# source code for the Meal Planning Interface, including methods for loading the meal master data and the user's daily plan, the logic for calculating daily macronutrient totals, and the event handlers for adding and removing planned meals using parameterized queries.

```
This includes:
    - Constructor initialization, setting the currentUserId, and initial data
    loads.
    - LoadMealsData: Populating the list of available food items (Meals
    table).
    - LoadUserPlan: Querying and aggregating data from the UserMealPlans and
    Meals tables.
    - btnAddToPlan_Click: Handling the INSERT operation with date and meal
    type parameters.
    - btnRemoveFromPlan_Click: Handling the DELETE operation based on the
    unique plan_id.
    - dtpDate_ValueChanged: Triggering a data reload when the plan date
    changes.

using MySql.Data.MySqlClient;
using System;
using System.Data;
using System.Drawing;
using System.Windows.Forms;

namespace FitnessPlanner
{
    public partial class FormMealPlanning : Form
    {
        private int currentUserId;
        private DataTable dtMeals;

        // Constructor receives the logged-in User ID
        public FormMealPlanning(int userId)
        {
            InitializeComponent();
            this.currentUserId = userId;

            // Set initial selected meal type
            if (cmbMealType.Items.Count > 0)
            {
```

```

        cmbMealType.SelectedIndex = 0;
    } else {
        // Initialize items if not done in designer
        cmbMealType.Items.AddRange(new object[] { "Breakfast",
"Lunch", "Dinner", "Snack" });
        cmbMealType.SelectedIndex = 0;
    }

    // Load initial data
    LoadMealsData();
    LoadUserPlan(dtpDate.Value.Date);
}

// DATA LOADING
private void LoadMealsData()
{
    // Select all meal details
    string query = "SELECT meal_id, name, calories, protein_g,
carbs_g, fat_g FROM Meals ORDER BY name";

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlDataAdapter adapter = new MySqlDataAdapter(query,
DBConnection.Instance().Connection);
            dtMeals = new DataTable();
            adapter.Fill(dtMeals);

            dgvAvailableMeals.DataSource = dtMeals;

            if (dgvAvailableMeals.Columns.Contains("meal_id"))
                dgvAvailableMeals.Columns["meal_id"].Visible = false;

            if (dgvAvailableMeals.Columns.Contains("name"))
            {
                dgvAvailableMeals.Columns["name"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;
                dgvAvailableMeals.Columns["name"].HeaderText = "Meal
Name";
            }

            // Set other columns to size based on content width

```

```

        if (dgvAvailableMeals.Columns.Contains("calories"))
dgvAvailableMeals.Columns["calories"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;
        if (dgvAvailableMeals.Columns.Contains("protein_g"))
dgvAvailableMeals.Columns["protein_g"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;
        if (dgvAvailableMeals.Columns.Contains("carbs_g"))
dgvAvailableMeals.Columns["carbs_g"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;
        if (dgvAvailableMeals.Columns.Contains("fat_g"))
dgvAvailableMeals.Columns["fat_g"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.AllCells;

        // Rename columns for display
        if (dgvAvailableMeals.Columns.Contains("calories"))
dgvAvailableMeals.Columns["calories"].HeaderText = "Kcal";
        if (dgvAvailableMeals.Columns.Contains("protein_g"))
dgvAvailableMeals.Columns["protein_g"].HeaderText = "Protein (g)";
        if (dgvAvailableMeals.Columns.Contains("carbs_g"))
dgvAvailableMeals.Columns["carbs_g"].HeaderText = "Carbs (g)";
        if (dgvAvailableMeals.Columns.Contains("fat_g"))
dgvAvailableMeals.Columns["fat_g"].HeaderText = "Fat (g)";
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error loading meal database: " +
ex.Message, "Database Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally
    {
        DBConnection.Instance().CloseConnection();
    }
}

// Loads the user's planned meals for a specific date
private void LoadUserPlan(DateTime date)
{
    string dateStr = date.ToString("yyyy-MM-dd");
    string query = @"
        SELECT up.plan_id, up.meal_type, m.name,
               (m.calories) AS TotalCalories,
               (m.protein_g) AS TotalProtein,
               (m.carbs_g) AS TotalCarbs,

```

```

        (m.fat_g) AS TotalFat
    FROM UserMealPlans up
    JOIN Meals m ON up.meal_id = m.meal_id
    WHERE up.user_id = @userId AND up.plan_date = @date
    ORDER BY up.plan_id"; // Order by plan ID to maintain
insertion order

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
            cmd.Parameters.AddWithValue("@userId", currentUserId);
            cmd.Parameters.AddWithValue("@date", dateStr);

            MySqlDataAdapter adapter = new MySqlDataAdapter(cmd);
            DataTable dtPlan = new DataTable();
            adapter.Fill(dtPlan);

            dgvUserPlan.DataSource = dtPlan;

            if (dgvUserPlan.Columns.Contains("plan_id"))
                dgvUserPlan.Columns["plan_id"].Visible = false;

            if (dgvUserPlan.Columns.Contains("meal_type"))
                dgvUserPlan.Columns["meal_type"].HeaderText = "Type";
            if (dgvUserPlan.Columns.Contains("name"))
                dgvUserPlan.Columns["name"].HeaderText = "Meal";
            if (dgvUserPlan.Columns.Contains("TotalCalories"))
                dgvUserPlan.Columns["TotalCalories"].HeaderText =
"Kcal";

            if (dgvUserPlan.Columns.Contains("TotalProtein"))
                dgvUserPlan.Columns["TotalProtein"].HeaderText =
"Protein (g)";

            if (dgvUserPlan.Columns.Contains("TotalCarbs"))
                dgvUserPlan.Columns["TotalCarbs"].HeaderText = "Carbs
(g)";

            if (dgvUserPlan.Columns.Contains("TotalFat"))
                dgvUserPlan.Columns["TotalFat"].HeaderText = "Fat
(g)";

            // Calculate and display total calories and
macronutrients

            int totalCal = 0;

```

```

        int totalProtein = 0;
        int totalCarbs = 0;
        int totalFat = 0;

        foreach (DataRow row in dtPlan.Rows)
        {
            if (row["TotalCalories"] != DBNull.Value) totalCal +=
Convert.ToInt32(row["TotalCalories"]);
            if (row["TotalProtein"] != DBNull.Value) totalProtein
+= Convert.ToInt32(row["TotalProtein"]);
            if (row["TotalCarbs"] != DBNull.Value) totalCarbs +=
Convert.ToInt32(row["TotalCarbs"]);
            if (row["TotalFat"] != DBNull.Value) totalFat +=
Convert.ToInt32(row["TotalFat"]);
        }

        // Update the summary label
        lblTotalCalories.Text = $"Total Macros: {totalCal} Kcal |
P: {totalProtein}g | C: {totalCarbs}g | F: {totalFat}g";
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error loading user plan: " + ex.Message,
"Database Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally
    {
        DBConnection.Instance().CloseConnection();
    }
}

// INTERACTION HANDLERS
// Adds the selected meal to the user's plan for the selected date
private void btnAddToPlan_Click(object sender, EventArgs e)
{
    if (dgvAvailableMeals.CurrentRow == null)
    {
        MessageBox.Show("Please select a meal from the database to
add.", "Selection Error", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    if (!dgvAvailableMeals.Columns.Contains("meal_id"))

```

```

        {
            MessageBox.Show("Internal Error: The 'meal_id' column was not loaded.", "Data Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        // 1. Get selected meal details
        int mealIdColumnIndex =
dgvAvailableMeals.Columns["meal_id"].Index;
        int mealId =
Convert.ToInt32(dgvAvailableMeals.CurrentRow.Cells[mealIdColumnIndex].Value);

        string mealType = cmbMealType.SelectedItem?.ToString();
        DateTime date = dtpDate.Value.Date;

        if (string.IsNullOrEmpty(mealType))
        {
            textBox1.Text = "Please select a Meal Type (Breakfast, Lunch,
etc.)";
            return;
        }

        // 2. Insert into UserMealPlans
        string query = "INSERT INTO UserMealPlans (user_id, plan_date,
meal_id, meal_type) VALUES (@userId, @date, @mealId, @mealType)";

        if (DBConnection.Instance().OpenConnection())
        {
            try
            {
                MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
                cmd.Parameters.AddWithValue("@userId", currentUserId);
                cmd.Parameters.AddWithValue("@date", date.ToString("yyyy-
MM-dd"));

                cmd.Parameters.AddWithValue("@mealId", mealId);
                cmd.Parameters.AddWithValue("@mealType", mealType);

                cmd.ExecuteNonQuery();

                // 3. Refresh the user's plan view
                LoadUserPlan(date);
            }
            catch (MySqlException ex) when (ex.Number == 1062) // MySQL
Duplicate entry error

```



```

        {
            textBox1.Text = "You have already planned this meal for
the selected time on this date.";
        }
        catch (Exception ex)
        {
            textBox1.Text = "Error adding meal: " + ex.Message;
        }
        finally
        {
            DBConnection.Instance().CloseConnection();
        }
    }
}

// Removes the selected meal entry from the user's plan
private void btnRemoveFromPlan_Click(object sender, EventArgs e)
{
    if (dgvUserPlan.CurrentRow == null ||
!dgvUserPlan.Columns.Contains("plan_id") ||
dgvUserPlan.CurrentRow.Cells["plan_id"].Value == DBNull.Value)
    {
        textBox2.Text = "Please select a meal from your daily plan to
remove.";
        return;
    }

    int planId =
Convert.ToInt32(dgvUserPlan.CurrentRow.Cells["plan_id"].Value);
    DateTime date = dtpDate.Value.Date;

    string query = "DELETE FROM UserMealPlans WHERE plan_id = @planId
AND user_id = @userId";

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
            cmd.Parameters.AddWithValue("@planId", planId);
            cmd.Parameters.AddWithValue("@userId", currentUserId); //
Added for extra security

            int rowsAffected = cmd.ExecuteNonQuery();

```

```

        if (rowsAffected > 0)
        {
            // Refresh the user's plan view
            LoadUserPlan(date);
        }
        else
        {
            textBox2.Text = "Removal failed. Meal not found or
user ownership mismatch.";
        }
    }
    catch (Exception ex)
    {
        textBox2.Text = "Error removing meal: " + ex.Message;
    }
    finally
    {
        DBConnection.Instance().CloseConnection();
    }
}

// Handles the date picker change event
private void dtpDate_ValueChanged(object sender, EventArgs e)
{
    // Reload the plan whenever the date changes
    LoadUserPlan(dtpDate.Value.Date);
}

private void dgvAvailableMeals_CellFormatting(object sender,
DataGridViewCellFormattingEventArgs e)
{
    if (dgvAvailableMeals.CurrentRow != null && e.RowIndex ==
dgvAvailableMeals.CurrentRow.Index)
    {
        e.CellStyle.BackColor = Color.FromArgb(230, 240, 255);
    }
}

private void lblUserPlan_Click(object sender, EventArgs e)
{
    // Unused event handler
}
}
}

```

Appendix F: Full Source Code for Workout Planning Interface

This appendix contains the complete C# source code for the Workout Planning Interface, detailing the methods for populating the exercise list, displaying the user's daily plan, and the logic for adding, removing, and marking a workout as completed.

```
this includes:
    - Constructor initialization with the currentUserId.
    - LoadAllWorkouts: Populating the list of available exercises (Workouts
table).
    - LoadUserWorkoutPlan: Querying and displaying the scheduled exercises
for the selected date.
    - dtpPlanDate_ValueChanged: Triggering a data reload when the plan date
changes.
    - btnAddToPlan_Click: Handling the INSERT operation to schedule an
exercise with Sets and Reps.
    - btnRemoveFromPlan_Click: Handling the DELETE operation to remove a
scheduled item.
    - btnMarkCompleted_Click: Logging a 'WorkoutCompleted' metric into the
UserProgress table with transaction control.

using MySql.Data.MySqlClient;
using System;
using System.Data;
using System.Windows.Forms;

namespace FitnessPlanner
{
    public partial class FormWorkoutPlanning : Form
    {
        private int currentUserId;

        public FormWorkoutPlanning(int userId)
        {
            InitializeComponent();
            this.currentUserId = userId;
        }

        private void FormWorkoutPlanning_Load(object sender, EventArgs e)
        {
            LoadAllWorkouts();
            LoadUserWorkoutPlan(dtpPlanDate.Value.Date);
        }

        // Data Loading
```

```

private void LoadAllWorkouts()
{
    string query = "SELECT workout_id, name FROM Workouts ORDER BY
name";
    DataTable dtWorkouts = new DataTable();

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
            MySqlDataAdapter adapter = new MySqlDataAdapter(cmd);
            adapter.Fill(dtWorkouts);

            // Set DataSource first
            dgvAllWorkouts.DataSource = dtWorkouts;

            // set column visibility and headers after binding
            if (dgvAllWorkouts.Columns.Contains("workout_id"))
            {
                dgvAllWorkouts.Columns["workout_id"].Visible = false;
            }

            if (dgvAllWorkouts.Columns.Contains("name"))
            {
                dgvAllWorkouts.Columns["name"].HeaderText = "Exercise
Name";
                dgvAllWorkouts.Columns["name"].Visible = true;
                dgvAllWorkouts.Columns["name"].AutoSizeMode =
DataGridViewAutoSizeColumnMode.Fill;
            }

            dgvAllWorkouts.Refresh();
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error loading available workouts: " +
ex.Message, "Database Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        finally
        {
            DBConnection.Instance().CloseConnection();
        }
    }
}

```

```

    }
}

private void LoadUserWorkoutPlan(DateTime date)
{
    string dateStr = date.ToString("yyyy-MM-dd");
    string query = @"
        SELECT up.plan_id, w.name, up.sets, up.reps
        FROM UserWorkoutPlans up
        JOIN Workouts w ON up.workout_id = w.workout_id
        WHERE up.user_id = @userId AND up.plan_date = @date
        ORDER BY w.name";

    DataTable dtPlan = new DataTable();

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
            cmd.Parameters.AddWithValue("@userId", currentUserId);
            cmd.Parameters.AddWithValue("@date", dateStr);

            MySqlDataAdapter adapter = new MySqlDataAdapter(cmd);
            adapter.Fill(dtPlan);

            dgvCurrentPlan.DataSource = dtPlan;

            // Column Header and Visibility Settings
            if (dgvCurrentPlan.Columns.Contains("plan_id"))
dgvCurrentPlan.Columns["plan_id"].Visible = false;
            if (dgvCurrentPlan.Columns.Contains("name"))
dgvCurrentPlan.Columns["name"].HeaderText = "Exercise";
            if (dgvCurrentPlan.Columns.Contains("sets"))
dgvCurrentPlan.Columns["sets"].HeaderText = "Sets";
            if (dgvCurrentPlan.Columns.Contains("reps"))
dgvCurrentPlan.Columns["reps"].HeaderText = "Reps";

            // Update title based on results
            if (dtPlan.Rows.Count == 0)
            {
                lblCurrentPlanTitle.Text = $"Your Plan for:
{date.ToShortDateString()} (Empty)";
            }
        }
        catch { }
    }
}

```

```

        }
        else
        {
            lblCurrentPlanTitle.Text = $"Your Plan for:
{date.ToShortDateString()}";
        }

        dgvCurrentPlan.Refresh();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error loading plan: " + ex.Message,
"Database Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally
    {
        DBConnection.Instance().CloseConnection();
    }
}

// Event Handlers

private void dtpPlanDate_ValueChanged(object sender, EventArgs e)
{
    LoadUserWorkoutPlan(dtpPlanDate.Value.Date);
}

private void btnAddToPlan_Click(object sender, EventArgs e)
{
    if (dgvAllWorkouts.SelectedRows.Count == 0)
    {
        textBox1.Text = "Please select an exercise from the list
below.";
        return;
    }

    Ensure the selected row is
    if (dgvAllWorkouts.SelectedRows[0].Cells["workout_id"].Value ==
null || dgvAllWorkouts.SelectedRows[0].Cells["workout_id"].Value ==
DBNull.Value)
    {
        textBox1.Text = "Selected row has no valid Workout. Please
select a different exercise.";
        return;
    }
}

```

```

    }

    int workoutId =
Convert.ToInt32(dgvAllWorkouts.SelectedRows[0].Cells["workout_id"].Value);
    int sets = (int)nudSets.Value;
    int reps = (int)nudReps.Value;
    string dateStr = dtpPlanDate.Value.Date.ToString("yyyy-MM-dd");

    string query = "INSERT INTO UserWorkoutPlans (user_id,
workout_id, plan_date, sets, reps) VALUES (@userId, @workoutId, @date, @sets,
@reps)";

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
            cmd.Parameters.AddWithValue("@userId", currentUserId);
            cmd.Parameters.AddWithValue("@workoutId", workoutId);
            cmd.Parameters.AddWithValue("@date", dateStr);
            cmd.Parameters.AddWithValue("@sets", sets);
            cmd.Parameters.AddWithValue("@reps", reps);

            cmd.ExecuteNonQuery();
            textBox1.Text = "Workout added to plan successfully!";
            LoadUserWorkoutPlan(dtpPlanDate.Value.Date); // Reload
the plan
        }
        catch (MySqlException ex) when (ex.Number == 1062) // MySQL
Duplicate entry error (Unique Index Violation)
        {
            textBox1.Text = "This exercise is already planned for
this date. Change the sets/reps or remove it first.";
        }
        catch (Exception ex)
        {
            textBox1.Text = "Error adding workout: " + ex.Message;
        }
        finally
        {
            DBConnection.Instance().CloseConnection();
        }
    }
}

```

```

        private void btnRemoveFromPlan_Click(object sender, EventArgs e)
        {
            if (dgvCurrentPlan.SelectedRows.Count == 0 ||
!dgvCurrentPlan.Columns.Contains("plan_id"))
            {
                textBox2.Text = "Please select a workout from your current
plan to remove.";
                return;
            }

            // Check if plan_id cell value is valid before conversion
            if (dgvCurrentPlan.SelectedRows[0].Cells["plan_id"].Value == null
|| dgvCurrentPlan.SelectedRows[0].Cells["plan_id"].Value == DBNull.Value)
            {
                textBox2.Text = "Error: Could not retrieve plan ID for
removal.";
                return;
            }

            int planId =
Convert.ToInt32(dgvCurrentPlan.SelectedRows[0].Cells["plan_id"].Value);

            // Directly proceed with deletion
            string query = "DELETE FROM UserWorkoutPlans WHERE plan_id =
@planId AND user_id = @userId";

            if (DBConnection.Instance().OpenConnection())
            {
                try
                {
                    MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
                    cmd.Parameters.AddWithValue("@planId", planId);
                    cmd.Parameters.AddWithValue("@userId", currentUserId); //
Security check

                    cmd.ExecuteNonQuery();
                    textBox2.Text = "Workout removed successfully!";
                    LoadUserWorkoutPlan(dtpPlanDate.Value.Date); // Reload
the plan
                }
                catch (Exception ex)
                {
                    textBox2.Text = "Error removing workout: " + ex.Message;

```



```

        return;
    }
    finally
    {
        DBConnection.Instance().CloseConnection();
    }
}
}
private void btnMarkCompleted_Click(object sender, EventArgs e)
{
    // 1. Check if the current plan date is today
    if (dtpPlanDate.Value.Date != DateTime.Today.Date)
    {
        textBox2.Text = "You can only mark the current day's workout
as completed. Please switch the date to today.";
        return;
    }

    // Check if the plan is actually populated
    if (dgvCurrentPlan.Rows.Count == 0 || (dgvCurrentPlan.Rows.Count
== 1 && dgvCurrentPlan.Rows[0].IsNewRow))
    {
        textBox2.Text = "No workouts planned for today. Nothing to
mark as complete.";
        return;
    }

    // 2. Log the completion to UserProgress
    string dateStr = DateTime.Today.ToString("yyyy-MM-dd");
    string metricType = "WorkoutCompleted";
    decimal value = 1; // 1 means completed

    string query = "INSERT INTO UserProgress (user_id, log_date,
metric_type, metric_value) VALUES (@userId, @date, @metricType, @value)";
    string checkQuery = "SELECT COUNT(*) FROM UserProgress WHERE
user_id = @userId AND log_date = @date AND metric_type = @metricType";

    if (DBConnection.Instance().OpenConnection())
    {
        MySqlTransaction transaction = null;
        try
        {
            transaction =
DBConnection.Instance().Connection.BeginTransaction();

```

```

        // Check if already logged
        MySqlCommand checkCmd = new MySqlCommand(checkQuery,
DBConnection.Instance().Connection, transaction);
        checkCmd.Parameters.AddWithValue("@userId",
currentUserId);

        checkCmd.Parameters.AddWithValue("@date", dateStr);
        checkCmd.Parameters.AddWithValue("@metricType",
metricType);

        long count = (long)checkCmd.ExecuteScalar();
        if (count > 0)
        {
            textBox2.Text = "Today's workout is already marked as
completed!";

            transaction.Rollback();
            return;
        }

        // Proceed with insertion
        MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection, transaction);
        cmd.Parameters.AddWithValue("@userId", currentUserId);
        cmd.Parameters.AddWithValue("@date", dateStr);
        cmd.Parameters.AddWithValue("@metricType", metricType);
        cmd.Parameters.AddWithValue("@value", value);

        cmd.ExecuteNonQuery();
        transaction.Commit();

        textBox2.Text = "Today's workout successfully marked as
completed!";

        // Close the form to return to the dashboard
        this.Close();
    }
    catch (Exception ex)
    {
        transaction?.Rollback();
        MessageBox.Show("Error logging workout: " + ex.Message,
"Database Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally
    {
        DBConnection.Instance().CloseConnection();
    }
}

```

```

    }
}
}

```

Appendix G: Full Source Code for Progress Interface

This documentation summarizes the progress tracking interface's functionality, covering logging, data visualization, and the BMI calculator, along with the full source code for reference.

```

using MySql.Data.MySqlClient;
using System.Data;
using System.Windows.Forms;
using System;
using System.Windows.Forms.DataVisualization.Charting;
using System.Drawing;

namespace FitnessPlanner
{
    public partial class FormProgress : Form
    {
        private int currentUserId;

        public FormProgress(int userId)
        {
            InitializeComponent();
            this.currentUserId = userId;

            // Set initial values for logging
            nudWeight.Value = 75;
            nudCaloriesOut.Value = 500;
        }

        private void FormProgress_Load(object sender, EventArgs e)
        {
            // Load both charts
            LoadAllProgressCharts();
        }

        //LOGGING SECTION -

```

```

private void btnLogWeight_Click(object sender, EventArgs e)
{
    decimal weight = nudWeight.Value;
    LogProgress("Weight", weight);
    LoadProgressData(chartWeight, "Weight"); //Refresh only the
Weight chart
}

private void btnLogCaloriesOut_Click(object sender, EventArgs e)
{
    decimal caloriesOut = nudCaloriesOut.Value;
    LogProgress("CaloriesOut", caloriesOut);
    LoadProgressData(chartCaloriesOut, "CaloriesOut"); // Refresh
only the Calories Out chart
}

private void LogProgress(string metricType, decimal value)
{
    // ON DUPLICATE KEY UPDATE in a real app.
    string query = "INSERT INTO UserProgress (user_id, log_date,
metric_type, metric_value) VALUES (@userId, @date, @metricType, @value)";

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
            cmd.Parameters.AddWithValue("@userId", currentUserId);
            cmd.Parameters.AddWithValue("@date",
DateTime.Today.ToString("yyyy-MM-dd"));
            cmd.Parameters.AddWithValue("@metricType", metricType);
            cmd.Parameters.AddWithValue("@value", value);

            cmd.ExecuteNonQuery();
            MessageBox.Show($"{metricType} logged successfully!",
"Success", MessageBoxButtons.OK, MessageBoxIcon.Information);
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error logging progress: " + ex.Message);
        }
        finally
        {

```

```

        DBConnection.Instance().CloseConnection();
    }
}

// VISUALIZATION SECTION

private void LoadAllProgressCharts()
{
    LoadProgressData(chartWeight, "Weight");
    LoadProgressData(chartCaloriesOut, "CaloriesOut");
}

private void LoadProgressData(Chart chartControl, string metricType)
{
    string query = "SELECT log_date, metric_value FROM UserProgress
WHERE user_id = @userId AND metric_type = @metricType ORDER BY log_date ASC";
    DataTable dtProgress = new DataTable();

    if (DBConnection.Instance().OpenConnection())
    {
        try
        {
            MySqlCommand cmd = new MySqlCommand(query,
DBConnection.Instance().Connection);
            cmd.Parameters.AddWithValue("@userId", currentUserId);
            cmd.Parameters.AddWithValue("@metricType", metricType);

            MySqlDataAdapter adapter = new MySqlDataAdapter(cmd);
            adapter.Fill(dtProgress);
        }
        catch (Exception ex)
        {
            // show on a bad query/connection, not if no data is
found
            MessageBox.Show($"Error loading {metricType} progress
data: " + ex.Message);
        }
        finally
        {
            DBConnection.Instance().CloseConnection();
        }
    }
}

```

```

        DisplayChart(chartControl, dtProgress, metricType);
    }

    private void DisplayChart(Chart chartControl, DataTable data, string
metricName)
    {
        // 1. Clear previous data
        chartControl.Series.Clear();
        chartControl.Titles.Clear();

        // 2. Set Chart Title
        chartControl.Titles.Add($"{metricName} Progress");

        // 3. Create a new Series (the line on the graph)
        Color chartColor = metricName == "Weight" ? Color.DodgerBlue :
Color.OrangeRed;

        Series series = new Series
        {
            Name = metricName,
            IsVisibleInLegend = true,
            ChartType = SeriesChartType.Line,
            Color = chartColor,
            BorderWidth = 3,
            MarkerStyle = MarkerStyle.Circle,
            MarkerSize = 6,
            MarkerColor = chartColor
        };

        // 4. Add data points
        foreach (DataRow row in data.Rows)
        {
            if (row["log_date"] != DBNull.Value && row["metric_value"] !=
DBNull.Value)
            {
                DateTime date = Convert.ToDateTime(row["log_date"]);
                double value = Convert.ToDouble(row["metric_value"]);
                series.Points.AddXY(date, value);
            }
        }

        // If no data, display a placeholder
        if (series.Points.Count == 0)
        {
            series.Points.AddXY(DateTime.Today.AddDays(-7), 0);
        }
    }

```

```

        series.Points.AddXY(DateTime.Today, 0);
        series.Color = Color.LightGray;
        series.BorderDashStyle = ChartDashStyle.Dash;
    }

    // 5. Add the series to the chart
    chartControl1.Series.Add(series);

    // 6. Configure Axis
    chartControl1.ChartAreas[0].AxisX.Title = "Date";
    chartControl1.ChartAreas[0].AxisX.MajorGrid.LineColor =
Color.LightGray;
    chartControl1.ChartAreas[0].AxisX.LabelStyle.Format = "MM-dd";
    chartControl1.ChartAreas[0].AxisX.LabelStyle.Angle = -45;

    chartControl1.ChartAreas[0].AxisY.Title = $"{metricName}
({(metricName == "Weight" ? "kg" : "kcal")}");
    chartControl1.ChartAreas[0].AxisY.MajorGrid.LineColor =
Color.LightGray;
    chartControl1.ChartAreas[0].AxisY.IsLabelAutoFit = true;
}

// BMI CALCULATOR SECTION

private void btnCalculateBmi_Click(object sender, EventArgs e)
{
    if (double.TryParse(txtWeightBMI.Text, out double weightKg) &&
double.TryParse(txtHeight.Text, out double heightM))
    {
        if (heightM > 0)
        {
            double bmi = weightKg / (heightM * heightM);
            string status = GetBmiStatus(bmi);

            lblBmiValue.Text = $"{bmi:0.00}";
            lblBmiStatusValue.Text = status;
            SetBmiStatusColor(status);
        }
        else
        {
            textBox1.Text = "Height must be greater than zero.";
        }
    }
    else
    {

```

```

        textBox1.Text = "Please enter valid numerical values for
Weight (kg) and Height (m).";
    }
}

private string GetBmiStatus(double bmi)
{
    if (bmi < 18.5) return "Underweight";
    if (bmi < 25.0) return "Healthy Weight";
    if (bmi < 30.0) return "Overweight";
    return "Obese";
}

private void SetBmiStatusColor(string status)
{
    switch (status)
    {
        case "Healthy Weight":
            lblBmiStatusValue.ForeColor = Color.ForestGreen;
            break;
        case "Underweight":
            lblBmiStatusValue.ForeColor = Color.DarkOrange;
            break;
        case "Overweight":
            lblBmiStatusValue.ForeColor = Color.OrangeRed;
            break;
        case "Obese":
            lblBmiStatusValue.ForeColor = Color.Red;
            break;
        default:
            lblBmiStatusValue.ForeColor = Color.Gray;
            break;
    }
}
}
}
}

```


7.References

[1]MyFitnessPal, Inc. “MyFitnessPal: Calorie Tracker & BMR Calculator to Reach Your Goals”. Internet: <https://www.myfitnesspal.com/> [Oct. 24, 2025]

[2]Strong Fitness PTE Limited. “Strong - Workout Tracker & Gym Log”. Internet: <https://www.strong.app/> [Oct. 24, 2025]

[3]Google Inc. “Google Fit: Activity Tracking - Apps on Google Play”. Internet: <https://play.google.com/store/apps/details?id=com.google.android.apps.fitness>, Oct. 2, 2025 [Oct. 24, 2025]

[4]Centers for Disease Control and Prevention (CDC). “Adult BMI Categories”. Internet: <https://www.cdc.gov/bmi/adult-calculator/bmi-categories.html>, Dec. 11, 2023 [Oct. 24, 2025]

Books

A. B. Chaudhuri, Flowchart and Algorithm Basics: The Art of Programming. Mercury Learning and Information, 2020.

- Citation Value: Primary source for flowcharting and pseudocode logic.

P. Bouna, Fundamentals of Programming Using Flowchart and Pseudocode. Packt Publishing, 2022.

- Citation Value: Modern, specialized reference for programming logic fundamentals.

J. Skeet, C# in Depth, 4th ed. Manning Publications, 2019.

- Citation Value: Authoritative source for advanced C# language features and principles.

J. Albahari and B. Albahari, C# 12 in a Nutshell. O'Reilly Media, 2024.

- Citation Value: Comprehensive reference for the C# language and the .NET platform.

R. B. Whitaker, The C# Player's Guide, 4th ed. Starbound Software, 2016.

- Citation Value: Excellent for citing clear, practical C# tutorials and introductory concepts.

J. Sharp, Microsoft Visual C# Step by Step. Microsoft Press, 2015.

- Citation Value: Credible, hands-on, official Microsoft source for C# and Visual Studio setup and usage.

Websites

Microsoft, “C# Guide - .NET managed language,” Microsoft Learn. Online

. Available: <https://learn.microsoft.com/en-us/dotnet/csharp/>. [Accessed: Oct. 2025].

- Citation Value: Definitive, official source for C# syntax, language features, and best practices.

Codecademy, “Pseudocode and Flowchart: Complete Beginner's Guide,” Codecademy Blog.

Online

. Available: <https://www.codecademy.com/article/pseudocode-and-flowchart-complete-beginners-guide>. [Accessed: Oct. 2025].

- Citation Value: Reference for formal rules and conventions of writing pseudocode and drawing flowcharts.

YouTube Video Links

freeCodeCamp, “Learn C# Programming – Crash Course with Mini-Projects,” YouTube, Jan. 14, 2025.